Declarative programming in Mercury

A guide to Mercury's declarative and operational semantics

MARK BROWN

Version 0.4, March 2023

Copyright © 2022–2023, YesLogic Pty. Ltd.
Copyright © 2023, The Mercury team.

This work is distributed under the Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0) license. To view a copy of the license, visit: https://creativecommons.org/licenses/by-sa/4.0/

Contents

Preface **v**

1 Introduction **1**
1.1 Purpose 1
1.2 Mercury programming in a nutshell 2
1.3 Notation 3
1.4 Outline of the guide 3

2 Declarative semantics by example **7**
2.1 First examples 7
2.2 Intended interpretations 9
2.3 Running example: queue ADT 10
2.4 Purity 11
2.4.1 Types, modes, and purity 11
2.4.2 Running example: queue declarations 12
2.4.3 Mode-dependent clauses 15
2.4.4 Case study: `string.append/3` 16
2.5 Partial correctness 17
2.6 Running example: queue implementation 19
2.7 Declarative debugging 22
2.8 Summary 24

3 First-order predicate calculus **27**
3.1 Overview 27
3.2 Syntax 28
3.3 Expressions and goals 30
3.4 Implicit quantification 33
3.5 Clauses 34
3.6 First-order logic with equality 35
3.7 Axioms 35
3.7.1 What are axioms? 35
3.7.2 Equality axioms 36
3.7.3 Clause soundness axioms 37

3.7.4 Combined clause soundness axioms 38
3.7.5 Clause completeness axioms 39
3.7.6 Predicate and function completion 40
3.7.7 Mode-determinism assertions 42
3.8 Classical semantics 43
3.8.1 Universes 43
3.8.2 Assignments 43
3.8.3 Interpretations 44
3.8.4 Models 45
3.9 Example 47
3.10 Philosophical remarks 50

4 Operational semantics 51
4.1 Overview 51
4.2 Queries 52
4.3 Substitutions 52
4.4 Unification 53
4.5 SLD resolution 55
4.6 SLD trees 58
4.7 Soundness and completeness 60
4.8 Operational incompleteness 61
4.9 Negation-as-failure 62
4.10 Committed-choice nondeterminism 63
4.11 Structural rules 65
4.12 What does SLD stand for? 66

5 The execution algorithm 67
5.1 Run-time unification 67
5.2 Term representation 68
5.3 Switches 69

6 Extensions 71
6.1 Higher-order code 71
6.2 Partial functions 73
6.3 Exceptions 74
6.3.1 Catching 74
6.3.2 Throwing 75
6.4 Types 76

7 Non-classical models 77

A Glossary index 79

B Source code listings 85
B.1 Running example: queue ADT 85

Preface

This guide started life as some notes and slides aimed at helping YesLogic developers learn the basic principles of logic programming. Most of the developers had plenty of experience in languages such as Haskell and Rust; the main barrier to learning logic programming was, I think, a lack of familiarity with much of the jargon, as well as much of the folklore. Some help was in order from the developers more experienced in Mercury. This document is intended to form part of that help.

The notes first evolved into an article, and then into the format it currently takes. YesLogic generously agreed to the release of the material, which becaome Version 0.4; it is currently maintained by the Mercury team.

This current version is already quite useful, there is still a lot of room for expansion. We could include more information about some existing topics, such as types, modes, determinism, and uniqueness, and we could cover additional topics such as the following.

- Modules and abstract data types.
- Minimal model semantics.
- Many-sorted algebras. In the current version we just map everything down to first-order, since part of the argument for declarative programming is that the models are relatively simple.
- User-defined equality and comparison.
- Partially instantiated data structures.
- ...

There is also room for more working examples, and for more bibliographical references.

Chapter 7 is not yet written, though I consider it important to draw people's attention to the work of Lee Naish and Harald Sondergaard in this area. Hopefully, this will be addressed in a future version.

Mark Brown
March 2023

Chapter 1

Introduction

1.1 Purpose

In the Formal Semantics chapter of the Mercury Language Reference Manual, the declarative semantics of Mercury is given in a single paragraph. Readers with sufficient background in logic programming would find the definition familiar: a predicate calculus theory with a "language" specified by the type declarations in the program, and a set of axioms derived from the "completion" of the program. Readers without that background, however, are likely to find this paragraph too opaque to be helpful.

The case is similar with the operational semantics, which is defined with reference to "SLDNF resolution". The vast majority of people who know what this means also already know logic programming, so this definition is not helpful for those for whom Mercury is their first logic programming language.

The challenge for readers is particularly difficult since existing resources on the predicate calculus tend to come in two forms:

1. Those that focus on logic as it pertains to logic programming.[1] While these do a good job at connecting the logic with an operational semantics (that is, giving the logic a computational interpretation) there is often relatively little focus on the completion semantics, which is how Mercury is defined.

2. Those that focus on classical logic in its own right.[2] While these generally offer a more complete picture of the logic, they do not usually discuss resolution, which is the computational mechanism used in logic programming. In addition, the level of mathematical rigor, while important, can obscure the issues most relevant to logic programming.

This guide aims to bridge the gap between theory and practice. It is intended for programmers who have some knowledge of Mercury and want a deeper understanding, but who are unable to derive much practical information from the resources

[1] For example, Apt, K.R., 1997. *From logic programming to Prolog*. London: Prentice Hall.
[2] For example, https://plato.stanford.edu/entries/logic-classical/.

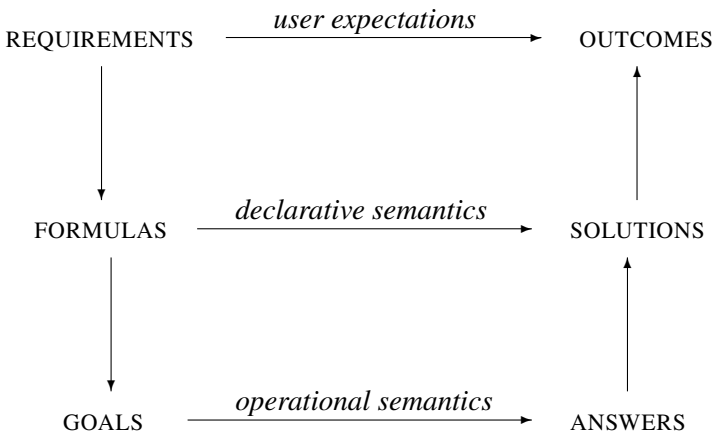

Figure 1.1: Mercury programming in a nutshell.

currently available. It is not presented with the same level of rigor as many other articles on this topic, for example, proofs are not provided for our claims. Rather, the intent is to put programmers in a better position to make the most practical use of existing resources.

1.2 Mercury programming in a nutshell

The main theme of this guide will be to show the parallels between syntax and semantics, of which there are many. By *syntax* we mean the sequences of characters that constitute part or all of a program. The word *semantics* means "meaning", but it also has a technical definition in the context of programming languages, which is that a program semantics constrains the program's behaviour with respect to a particular set of observables.

Figure 1.1 gives a conceptual view of the programming process in Mercury. At a high level, a programmer is given requirements, and in some way or other they need to generate outcomes in accordance with user expectations. They formulate the requirements logically, and with this formulation they can use the declarative semantics to determine what solutions—assignments of values to variables—arise as a consequence. These solutions are then interpreted in terms of the original problem domain, to generate outcomes that (hopefully) satisfy the user.

At a lower level, the programmer's mental formulation is expressed as goals in Mercury. The compiler and runtime system compute answers to the goals in a manner determined by the operational semantics, and these answers can in turn be understood by the programmer as solutions to the formulas.

These two levels of reasoning, the first more abstract and the second more concrete, are represented by the two horizontal arrows in the middle and in the lower part of the diagram. At each level there is a syntax to express the ideas and a se-

mantics to reason about what they mean. A close correspondence exists between the two, in that they place the same constraints on a program's observed behaviour.

The difference between the two levels of reasoning, and the reason we would want to consider having two distinct levels in the first place, comes down to how they are defined. The declarative semantics is defined in terms of semantic concepts understood by the programmer, and aims to characterize the programmer's mental picture of how a program behaves. On the other hand, the computer does not have such a mental picture—it blindly manipulates symbols without understanding how the programmer will interpret the results—so the operational semantics aims to characterize the program's behaviour as symbolic manipulation. Thus, the operational semantics is defined in terms of the syntax.

It is the declarative and operational semantics, and the correspondence between them, that is the principal subject of this guide.

1.3 Notation

With the dual syntax vs. semantics view in mind, we adopt a kind of parallel notation and terminology in this guide. When discussing program elements from a primarily syntactic or operational point of view, we will use Mercury syntax written in `this` font, whereas when the discussion is from a semantic or declarative point of view we will use conventional mathematical notation in normal font. Similarly, the terminology used differs between the two views, with terms that apply to the declarative view having their counterparts in the operational view. Figure 1.2 shows some examples of notation and terminology.

We hope that this use of notation will help readers. However, while the distinction it conveys between the semantic/declarative view on the one hand and the syntactic/operational view on the other hand is often useful, in some cases, it is not. Indeed, we will shortly introduce so-called Herbrand interpretations, in which elements of syntax are used directly as elements of the semantic domain.

1.4 Outline of the guide

The outline of the remainder of this guide is as follows.

In Chapter 2 we give an informal picture of the declarative semantics, with a focus on some simple examples. We aim to give a basic idea of what is meant by declarative semantics, and also discuss some of the advantages that can be obtained by thinking about Mercury programs in this way. This provides our motivation for wanting a declarative semantics.

In Chapter 3 we define the syntax and semantics of first-order predicate calculus, and show how the declarative semantics of a Mercury program is expressed in a predicate calculus theory. We give some examples of logical reasoning that can tell us how our programs behave.

Semantic/declarative	**Syntactic/operational**
variables:	variables:
x	`X`
$y_1, \ldots, y_n$	`Y1, ..., YN`
terms:	expressions:
$f(x, y)$	`f(X,Y)`
$\mathrm{t}_1, \ldots, \mathrm{t}_n$	`t1, ..., tN`
ground data terms:	constant expressions
123	`123`
$[1, 2, 3]$	`[1,2,3]`
$\mathrm{f}(\mathrm{c}, \mathrm{g}(\mathrm{d}))$	`f(c,g(d))`
$[1, 2, 3]$	`[1,2,3]`
$\mathrm{c}_1, \ldots, \mathrm{c}_n$	`c1, ..., cN`
atomic formulas:	atomic goals:
$y = \mathrm{f}(x)$	`Y = f(X)`
$\mathrm{p}(\mathrm{t}_1, \ldots, \mathrm{t}_n)$	`p(t1, ..., tN)`
logical connectives:	operators:
$\wedge$	`,`
$\vee$	`;`
$\leftarrow$	`:-`

Figure 1.2: Examples of the parallel notation we will use. The elements themselves will be discussed in later chapters.

In Chapter 4 we give abstract algorithms for unification and resolution, and use these as building blocks to define the operational semantics. We also define the negation-as-failure rule that is used to implement negation and if-then-else. We then give some important results in the meta-theory, which we use to show the correspondence between the operational and the declarative viewpoints.

In Chapter 5 we give concrete details of how the implementation behaves at run-time. These details should enable programmers to better estimate how different coding choices affect such operational characteristics of their programs as stack and heap usage.

In Chapter 6 we extend our work to cover aspects of Mercury which are not well-characterized in the classical semantics, such as partial functions and exceptions.

In Chapter 7 we present a non-classical interpretation of Mercury programs that is useful for writing specifications and checking that they are correctly implemented. This interpretation demonstrates that classical logic is not the only logic that can be usefully applied to understanding Mercury programs.

Finally, the glossary index in Appendix A provides short definitions, as well as page references, for many of the concepts discussed in the guide.

Chapter 2

Declarative semantics by example

2.1 First examples

Consider the code in Figure 2.1 that defines versions of length and append that are specialized to operate on lists whose elements belong to a very simple type that has only two values: `c` and `d`.

```
:- type e
    --->    c
    ;       d.

:- func len(list(e)) = int.

len([]) = 0.
len([_ | Xs]) = 1 + len(Xs).

:- pred app(list(e), list(e), list(e)).
:- mode app(in, in, out) is det.

app([], Bs, Bs).
app([X | As], Bs, [X | Cs]) :-
    app(As, Bs, Cs).
```

Figure 2.1: Specialized versions of length and append.

Figure 2.2 gives a picture of the declarative semantics of len/1, which consists of a table of *atomic formulas*, or *atoms* for short. (This is one of several uses of the word "atom" in logic programming; there are others as well.) For a function of arity n, each atom in the table consists of an equation between the function name applied to n argument terms, and a term that is the result of applying that function to the argument terms. So for len/1, this is an equation between len applied to a list, and the length of that list.

The atoms in the table must be *ground*, which means that they may not contain any variables. The table is infinite, but like the multiplication table—which is also infinite—it is relatively easy to get a picture of what is going on by looking at (or thinking about) only a finite part of it.

```
len([]) = 0                len([c, c, c]) = 3
len([c]) = 1               len([c, c, d]) = 3
len([d]) = 1               len([c, d, c]) = 3
len([c, c]) = 2            len([c, d, d]) = 3
len([c, d]) = 2            len([d, c, c]) = 3
len([d, c]) = 2            len([d, c, d]) = 3
len([d, d]) = 2            ...
```

Figure 2.2: Interpretation of len/1.

Similarly, Figure 2.3 gives a picture of the declarative semantics of app/3. For a predicate of arity *n*, each atom in the table consists of the predicate name applied to *n* terms. In this case the atom is not an equation and there is no result term, rather, it states that the arguments satisfy the relation denoted by that predicate. In the case of app/3 there are three list arguments, the third of which is the result of appending the first two.

```
app([], [], [])            app([], [c, d], [c, d])
app([], [c], [c])          app([c], [d], [c, d])
app([c], [], [c])          app([c, d], [], [c, d])
app([], [d], [d])          app([], [d, c], [d, c])
app([d], [], [d])          app([d], [c], [d, c])
app([], [c, c], [c, c])    app([d, c], [], [d, c])
app([c], [c], [c, c])      app([], [d, d], [d, d])
app([c, c], [], [c, c])    ...
```

Figure 2.3: Interpretation of app/3.

Tables like these are known as *Herbrand interpretations* (named after Jacques Herbrand, a French mathematician of the early 20th century). They assign a meaning to each predicate or function using a mapping from ground atoms to truth values: if a ground atom is in the table, then it is true; if it is not in the table, then it is false. For example, if we interpret the function symbol '+' as integer addition, then the table will contain entries such as `1 + 1 = 2`. On the other hand, if we interpret it as string concatenation, then the table will contain entries such as `"1" + "1" = "11"`.

Herbrand interpretations are purely syntactic in nature. This reflects the compiler's view of the program: the compiler does not know that len/1 is supposed to mean "list length", it only knows it as a symbol. These interpretations also reveal an important fact: the declarative semantics of a program can be understood in its entirety by considering only the truth values taken by ground atoms. That is, there

is no need to consider terms that include variables from the program.

Furthermore, this way of thinking works not just for tiny predicates like len/1 and app/3, but also for much larger pieces of code, with far more complexity in their intended interpretations. This ability to *scale* is crucial for extending our methodology to real-world programs.

2.2 Intended interpretations

An interpretation, generally, is something that allows the programmer to comprehend the meaning of terms, and to determine the truth of ground atoms, with reasonable ease. Essentially, it is a specification. If an interpretation reflects what the programmer intends to implement, it is called the *intended interpretation*.

To a programmer, even an informal definition can be sufficient. For example, we could give the intended interpretations of len/1 and app/1 as follows:

$$\text{for } n \geqslant 0, \text{len}([x_1, \ldots, x_n]) = n$$

$$\text{for } 0 \leqslant m \leqslant n, \text{app}([x_1, \ldots, x_m], [x_{m+1}, \ldots, x_n], [x_1, \ldots, x_n])$$

From these statements, it should be clear whether a particular ground atom is true or false in these interpretations, even though the interpretations are infinite in size. The truth value of formulas more generally can be determined by combining the truth values of atoms according to the truth tables of classical logic.

It is the usual practice in Mercury to describe the intended interpretation (along with any other pertinent information) in comments immediately preceding a `pred` or `func` declaration. For example, in the list module of the Mercury standard library, the declarations for the length/1 function and the length/2 predicate appear as follows:

```
    % length(List) = Length:
    % length(List, Length):
    %
    % True iff Length is the length of List, i.e. if
    % List contains Length elements.
    %
:- func length(list(T)) = int.
:- pred length(list(_T), int).
```

(The word “iff” is the usual abbreviation used in mathematics for “if and only if”.) Similarly, append/3 is described as follows:

```
    % Standard append predicate:
    % append(Start, End, List) is true iff
    % List is the result of concatenating Start and End.
    %
:- pred append(list(T), list(T), list(T)).
```

It should be easy to see that these are equivalent to the intended interpretations we gave above.

It is a good idea to provide comments that describe the intended interpretation of any function or predicate declared in the interface of a module. Doing this enables users of the module to understand whether or not they are using the interface correctly.

We saw above that a list such as `[1,2,3]` can be given the intended interpretation $[1, 2, 3]$. This might seem trivial, but it is worth noting that the former is meant to represent a piece of Mercury syntax, while the latter is meant to be the kind of notation that might be seen in a semi-formal mathematical proof. In this semi-formal notation, it is reasonable to use ellipses and subscripts, or other ad hoc notation, to describe the list structures.

A basic example for lists is the "cons" function, which takes an element and a list, and returns a new list with the element prepended. Since we do not need to know anything about the element we can just call it x, and we can assume the list takes the form $[t_1, \ldots, t_n]$, for some $n \geqslant 0$, and arbitrary elements t_i. We can therefore say that the intended interpretation of cons is a function that, given element x and list $[t_1, \ldots, t_n]$, returns the list $[x, t_1, \ldots, t_n]$.

We have already given intended interpretations for the list append predicate, and for the list length function. The list append function can be specified as:

$$\text{append}([s_1, \ldots, s_m], [t_1, \ldots, t_n]) = [s_1, \ldots, s_m, t_1, \ldots, t_n]$$

Similarly, the list reverse function can be specified as:

$$\text{reverse}([t_1, \ldots, t_n]) = [t_n, \ldots, t_1]$$

We will use these interpretations next when we implement the queue ADT.

2.3 Running example: queue ADT

In this section we give the intended interpretation for a (double-ended) queue abstract data type (*ADT*). We will use this as a running example in the remainder of this guide.

The ADT includes abstract operations to initialize a queue, put elements at the back and get them from the front, unput elements from the back and unget them at the front, test two queues for equality, and convert the queue to an ordinary list. A queue can be interpreted as a finite sequence of elements, using the same semi-formal mathematical notation as for lists.

The initialization function, init/0, is easy to specify because it just returns (a representation of) the empty queue:

$$\text{init} = []$$

Putting an element at the end of the queue is done with the put/3 predicate, which takes as arguments an element, the queue prior to putting the element at the end, and the queue after this has been done:

$$\text{put}(z, [x_1, \ldots, x_n], [x_1, \ldots, x_n, z])$$

Getting an element from the front of the queue is done with a get/3 predicate, which is similar except that the second argument is the queue prior to getting the element from the front:

$$\text{get}(z, [z, x_1, \ldots, x_n], [x_1, \ldots, x_n])$$

The inverse operations, unput/3 and unget/3, are the same as the forward operations except that the second and third arguments are reversed:

$$\text{unput}(z, [x_1, \ldots, x_n, z], [x_1, \ldots, x_n])$$
$$\text{unget}(z, [x_1, \ldots, x_n], [z, x_1, \ldots, x_n])$$

Finally, both the predicate eq/2 that tests if two queues contain the same sequence of elements, and the function list/1 that converts a queue to an ordinary list, are trivial to specify:

$$\text{eq}([x_1, \ldots, x_n], [x_1, \ldots, x_n])$$
$$\text{list}([x_1, \ldots, x_n]) = [x_1, \ldots, x_n]$$

That is, the intended interpretation of eq/2 is the identity relation, and the intended interpretation of list/1 is the identity function.

When implementing queues according to this specification, the queue type may be defined differently to the list type, though they are both interpreted as sequences of elements. If indeed they are different, the implementation of list/1 will not be trivial like the above specification, but will have to convert between the two representations. Similarly, if the representation is not *canonical*, meaning that the same queue may be represented in different ways, then the implementation of eq/2 will not be trivial.

2.4 Purity

2.4.1 Types, modes, and purity

The code we have seen so far is considered *pure*. Code that is written using all but a few Mercury constructs is automatically pure—most Mercury code is pure without the programmer needing to think about it. We give the following definition of what exactly we mean by purity.

Definition 1 (Purity)**.** *A predicate or function is* pure *if there exists a declarative interpretation describing its behaviour, that consistently applies in all circumstances.*

```
:- pred append(list(T), list(T), list(T)).
:- mode append(di, di, uo) is det.
:- mode append(in, in, out) is det.
:- mode append(in, in, in) is semidet.    % implied
:- mode append(in, out, in) is semidet.
:- mode append(out, out, in) is multi.
```

Figure 2.4: Declarations for append/3 in the list module of the standard library.

In other words, if for a given function or predicate, we can picture in our minds a Herbrand interpretation that characterizes the outcome of all possible calls, then that predicate or function is pure.

In the last section we gave the `pred` declaration for append/3 as it appears in the list module of the standard library. Figure 2.4 gives this `pred` declaration again, along with all of the declared modes.

The role of the `pred` declaration, which supplies the argument types, should be clear as regards the interpretations we have seen so far: a ground atom that is true in the interpretation will have arguments that are correctly typed. (If the types of the arguments include one or more type variables, then the arguments in the interpretation will have arguments that are correctly typed for some assignment of values—concrete types—to the type variables.) The compiler is able to check that this is the case.

The role of the `mode` declarations is perhaps not so clear in the declarative semantics, but in conjunction with the determinism each one constrains the set of ground atoms that are true, for the predicate or function as a whole. Importantly, and in line with our definition above, the same interpretation applies to *all* of the modes. In Mercury, a predicate or function is pure only if it has a consistent interpretation across all calls, regardless of the mode in which the call is made.

This means, for example, that if a call to append($[1], [2, 3], z$) yields a solution $z = [1, 2, 3]$, which it does, then a call to append($x, y, [1, 2, 3]$) should at some stage yield the solution $x = [1], y = [2, 3]$, which it also does. Both solutions derive from the fact that append($[1], [2, 3], [1, 2, 3]$) is true in the declarative interpretation. The same applies for any other pair of calls to different modes of append/3.

It should not be surprising that this is the case. The declarative semantics determines which solutions will be produced, and it in turn is determined by the clauses that define a predicate or function. Since the same clauses apply to all modes, no discrepancy can arise.

2.4.2 Running example: queue declarations

Continuing with our queue example, we can work out from our intended interpretation of the queue operations, as well as from our intended way of calling them, what the types, modes, and determinism of each operation will be. For reference,

$$\begin{aligned}
&\text{init} = [] \\
&\text{put}(z, [x_1, \ldots, x_n], [x_1, \ldots, x_n, z]) \\
&\text{get}(z, [z, x_1, \ldots, x_n], [x_1, \ldots, x_n]) \\
&\text{unput}(z, [x_1, \ldots, x_n, z], [x_1, \ldots, x_n]) \\
&\text{unget}(z, [x_1, \ldots, x_n], [z, x_1, \ldots, x_n]) \\
&\text{eq}([x_1, \ldots, x_n], [x_1, \ldots, x_n]) \\
&\text{list}([x_1, \ldots, x_n]) = [x_1, \ldots, x_n]
\end{aligned}$$

Figure 2.5: Intended interpretations for the queue operations

the intended interpretations are collected together in Figure 2.5.

Three types are used in the queue interface: the type of elements, the type of queues, and the type of lists. Since the behaviour of queues does not depend on the specific element values, we should be able to use elements of any type. Since we intend to treat all elements in the queue the same way, we require that all of the elements in a queue are the same type, and the same is true for the lists we produce (for which we will use the standard list/1 type).

We represent the type of elements with a *type variable* in the predicate/function declarations of the queue operations. Any actual calls to these predicates and functions will operate on elements of a specific concrete type, such as int or string. You can think of this as having the value of the type variable bound to the actual concrete type of the elements, for the duration of each call, just as e.g. the variable representing the first input argument is bound to the value being passed in that argument position in the call for the same duration.

Most Mercury code uses `T`, the initial letter of *type*, as the default name of type variables in type, predicate and function declarations that include only one type variable, and it is reasonably obvious what entity's type the variable stands for. (When a type, predicate or function declaration includes more than one type variable, or the role of the type variabl is not expected to be immediately clear, Mercury programmers tend to give the type variable(s) more meaningful names.)

Therefore the abstract type definition for queues will look like this:

```
:- type queue(T).
```

The lists will be of type `list(T)`, the definition of which is imported from the standard library using '`:- import_module list`'.

The init/0 function has no arguments, and always returns the same queue every time it is called. So the return value has mode `out`, and the function has determinism `det`. These are the default mode and default determinism for functions, so the declaration is:

```
:- func init = queue(T).
```

The four main operations have three arguments, the first being an element and the second and third being queues, with the elements in all three arguments being of the same type. Their `pred` declarations are therefore

```
:- pred put(T, queue(T), queue(T)).
:- pred get(T, queue(T), queue(T)).
:- pred unput(T, queue(T), queue(T)).
:- pred unget(T, queue(T), queue(T)).
```

For some of these operations the first argument is an input, and for others it is an output. The second and third arguments form an '`in`, `out`' pair—this is a useful pattern to follow since it allows for the convenient use of state variables. We intend the main operations to produce at most one result, even though there may be multiple representations of the same output value, so the determinism of each operation should be either `det` or `semidet`, depending on whether or not it always produces a solution.

For both the put/3 and unget/3 operations, the length of the output queue is one greater than the length of the input queue. There is no upper limit on the length of queues so these operations always have a solution, thus their determinism is `det`. For the get/3 and unput/3 operations, on the other hand, the length of the output queue is one *less* than the length of the input queue, which means there are no solutions if the input queue is empty. These operations must therefore be `semidet`, which leaves us with the following mode declarations:

```
:- mode put(in, in, out) is det.
:- mode get(out, in, out) is semidet.
:- mode unput(out, in, out) is semidet.
:- mode unget(in, in, out) is det.
```

The eq/2 predicate takes two queues and either succeeds or fails. It has no outputs, so it cannot produce multiple solutions and its determinism is therefore `semidet`. It is declared as follows:

```
:- pred eq(queue(T), queue(T)).
:- mode eq(in, in) is semidet.
```

Normally, programmers put the mode declaration(s) for a predicate or function immediately after the predicate or function declaration itself, as in this case. We separated the predicate declarations of put/3, get/3, unput/3 and unget/3 above from their mode declarations only to improve the flow of their explanations.

It is not an issue for these operations, but for predicates and functions with many arguments, programmers may find it hard to visually match up the mode of each argument with its type. This is why, in the usual case that a predicate or function has only one mode declaration, programmers tend to combine that mode declaration with the predicate or function declaration:

```
:- pred eq(queue(T)::in, queue(T)::in) is semidet.
```

We call this combination a *predmode* declaration, because in most cases, it is applied to predicates. For functions, there is usually no need for this combination, because most functions have the default argument modes and the default determinism, which do not need to be explicitly declared at all.

The last queue operation, the list/1 function, takes a queue and returns a list, and like init/0 it uses the default function mode:

```
:- func list(queue(T)) = list(T).
```

The overall set of declarations for the queue operations is therefore:

```
:- func init = queue(T).
:- pred put(T::in, queue(T)::in, queue(T)::out) is det.
:- pred get(T::out, queue(T)::in, queue(T)::out) is semidet.
:- pred unput(T::out, queue(T)::in, queue(T)::out) is semidet.
:- pred unget(T::in, queue(T)::in, queue(T)::out) is det.
:- pred eq(queue(T)::in, queue(T)::in) is semidet.
:- func list(queue(T)) = list(T).
```

2.4.3 Mode-dependent clauses

It can be useful to define multi-moded predicates such as append/3, the use of which can readily enable larger multi-moded predicates to be implemented. However, it can sometimes be the case that, irrespective of how the clauses defining a predicate are written, making one mode efficient inevitably results in another mode being inefficient. An example of this is the following mode of append/3, which is commented out in the standard library.

```
% :- mode append(out, in, in) is semidet.
```

The explanation for why it is commented out says that the mode "is `semidet` in the sense that it does not succeed more than once—but operationally, it does create a choice-point, which means both that it is inefficient, and that the compiler can't deduce that it is `semidet`. Use `remove_suffix` instead."

If such a mode is nonetheless required for a predicate, and the inefficiency would otherwise be unacceptable, then one solution is to implement the predicate using mode-dependent clauses, also known as "different clauses for different modes". Doing this, however, would invalidate our claim from the previous section that the declarative semantics is consistent across modes, since the meaning is no longer determined from a single set of clauses. If the clauses for different modes express different relations, then a consistent declarative semantics cannot exist.

So this is an issue of purity. Having a consistent declarative semantics is required, yet there is no way for the compiler to verify, in general, that two different sets of clauses express the same relation. As such, the compiler will treat a predicate defined using mode-dependent clauses as impure, unless the programmer is prepared to promise otherwise.

In the Prolog literature, impure predicates—those that do not have a consistent declarative semantics—are typically referred to as "non-logical". (Note that in Section 3.2, we will see this term used in a different sense.) The canonical example of a non-logical predicate in Prolog is `var/1`, which succeeds if and only if the argument is not bound. This can be implemented in Mercury as follows.

```
:- impure pred var(T).
:- mode var(in) is failure.
:- mode var(unused) is det.

var(_::in) :- false.
var(_::unused) :- true.
```

A call to `var(X)` would succeed if, at the point of call, `X` was not bound to anything. This implies that the predicate is interpreted as true for *every* possible argument value. If the same call was made with `X` bound to `a`, however, then the call would fail. This implies that the predicate is interpreted as false for the value a, which contradicts the previous implication. Thus it can be seen that the predicate is not pure according to our definition, since there does not exist a declarative interpretation that applies to all modes.

2.4.4 Case study: `string.append/3`

In this section we give an example of using the declarative semantics to resolve a problem related to the modes of a predicate. The problem arose in the course of developing the Mercury standard library, and involved the append/3 predicate in the string module.

In early versions of the Mercury, strings were defined as NUL terminated sequences of ASCII characters. As with lists, the string append/3 predicate had a "forward" mode that appended strings, as well as a "reverse" mode that split them apart. A consistent declarative semantics applied to both of these modes, since appending and splitting these strings were inverse operations.

When we switched to Unicode, using the UTF-8 representation when generating C code, a new situation arose. UTF-8 strings consist of a sequence of fixed-size code units, which are grouped together into code points that each contain a varying number of code units. It is possible for sequences of code units to be ill-formed, in particular if strings can be split between arbitrary code units.

In the forward (`in`, `in`, `out`) mode of the Unicode version of string append/3, the behaviour is to append the sequences of code units representing the (possibly ill-formed) input strings. Consider two strings s_1 and s_2, neither of which is well-formed, but which, when appended together, result in a string, call it s_3, which *is* well-formed.

Given that the forward mode of string append/3, when given s_1 and s_2 as the first two arguments, will return s_3 as the third argument, its intended interpretation must include ground atoms such as $\text{append}(s_1, s_2, s_3)$.

If we want the reverse (`out`, `out`, `in`) mode of string append/3 to have the same semantics as the forward mode, and we do, then the reverse mode of the predicate *must* return the pair s_1 and s_2 as one the solutions when given as input s_3 in the third argument position. This means that the reverse mode of string append/3 must split its input string at all positions between *code units*, including those in the *middle* of a code point, even though most callers of the predicate in this mode would expect that it would split the string only between *code points*, at least if the input string is well formed.

This means that we had to choose one of the following courses of action.

- We can make the (`out`,`out`,`in`) mode of string append/3 break strings between code points, which conforms to programmer expectations, but gives the reverse mode a different interpretation, i.e. a different semantics, than the forward mode.
- We can make the (`out`,`out`,`in`) mode of string append/3 break strings between code unit, which keeps the semantics the same between modes but breaks programmer expectations, and does so in a way that is likely to create bugs (by creating two ill-formed strings out of a well-formed input string).
- We can delete the (`out`,`out`,`in`) mode of string append/3. This does not break either the declarative semantics or programmer expectations, but it does have the downside of breaking existing code that calls that predicate in that mode.

We chose the third approach as the best one available. We deleted the reverse (`out`,`out`,`in`) mode of string append/3, and created nondet_append/3, a new predicate in the string module, to replace it. This new predicate has only one mode, the (`out`,`out`,`in`) mode, and it splits its input string between code points (provided the input string is itself well-formed). This does break backward compatibility, but it does so visibly: any old code that calls string append/3 in its now-deleted mode it would get a compiler error, until the call site is updated to call nondet_append/3 instead.

2.5 Partial correctness

Clauses are supposed to state things that are true in the intended interpretation. For example, in Figure 2.1 the first clause of len/1 states that the length of the empty list is zero. The second states that, no matter what terms we substitute for the variables `_` and `Xs`, the length of `[_ | Xs]` will be one greater than the length of `Xs`—in other words the length of a non-empty list is one greater than the length of its tail. Both statements are true according to our intended interpretation.

Furthermore, the clauses cover every possible list, in the sense that every list is either empty or non-empty, and every non-empty list has a tail that is also a list. Perhaps surprisingly, this is enough to conclude that our implementation is correct, at least as far as arguments and return values are concerned.

Our argument here depends on two points, which may be regarded as two sides of the same coin:

- Each of the clauses defining the function is a valid statement, where by valid we mean that every instance of the clause is true in the intended interpretation. This ensures that there are no "wrong answers": ground atoms that are intended to be false, but are true according to the program as written. We refer to this condition as *clause soundness*.

- Between them, the clauses cover every possible ground atom that is true in the intended interpretation. This ensures that there are no "missing answers": ground atoms that are intended to be true, but are false according to the program as written. We refer to this condition as *clause completeness*.

The two classes of bugs being avoided here, wrong answers and missing answers, are the bugs that are observable in the (classical) declarative semantics.

It is worth noting that, in many cases, Mercury's mode and determinism systems can make the compiler check the coverage for us: if the determinism indicates that calls cannot fail (that is, if the determinism is `det`, `multi`, or `cc_multi`), then all possible values of the type must be covered for any argument with mode `in`. In the case of the len/1 function, the one argument is an input because this is the default mode for functions, and this mode of the function is `det` because this is the default determinism for functions in their default mode. Had we not covered every possible list, the compiler would have issued a determinism error, so we can safely assume that the definition covers all possible solutions. The same also applies to any other function declared with the default mode.

Continuing with the examples, the first clause of app/3 is a fact, which states that if you append the empty list and any other list, the result will be the same as the other list. The second clause is a rule; these are taken as logical implications in which the body implies the head (that is, `:-` is interpreted as $\leftarrow$). So this clause is stating that, for any variable assignment, if `Cs` is the result of appending `As` and `Bs`, then `[X | Cs]` is the result of appending `[X | As]` and `Bs`. Again, both clauses are valid according to the intended interpretation, and the definition is clause complete. In this case clause completeness means that, for every atom that is intended to be true, there is either a fact that covers it, or a rule whose head covers it and whose body is intended to be true (under the same variable assignment).

Since both conditions are satisfied, we can conclude in a similar way to len/1 that our implementation of app/3 is correct.

We have been saying "correct" here, but this is only as far as the arguments (and return values, if present) are concerned, which is to say that there are neither wrong nor missing answers. Other kinds of bugs, such as unintended exceptions or nontermination, poor computational complexity, or unbounded stack usage, are not observable in the declarative semantics. We therefore refer to correctness in the above sense as *partial correctness*. This is akin to type correctness, in the sense that it rules out a certain class of bugs, but cannot rule out all bugs.

The results of this section can be summarized with a theorem.

Theorem 1 (Partial correctness)**.** *If every clause in a program is true in the intended interpretation and every predicate and function definition is clause complete according to this interpretation, then the program is partially correct.*

A corollary of this is that if a program is not partially correct, that is, if it produces wrong answers or misses answers that it should have produced, then there is at either least one clause in the program with an instance that is not true in the intended interpretation, or at least one definition that is not clause complete.

2.6 Running example: queue implementation

Armed with the notion of partial correctness, we are in a position to attempt an implementation of the queue ADT, and make a judgement as to whether it implements the intended interpretation.

We first need to choose an implementation for the queue/1 type. One technique is to use a pair of lists representing the front portion and the back portion of a queue. The queue is split at an arbitrary point so the two portions can be any length, including zero, and the back portion is stored in reverse order. In this way, the head of each list corresponds to one end of the queue or the other, which are the positions in the queue that our interface allows access to.

The type can be defined as follows:

```
:- type queue(T)
    --->    queue(
                front    :: list(T),
                rev_back :: list(T)
            ).
```

To formalize the above description, we can give the intended interpretation of the queue/2 data constructor. Although it is just a symbol and not a function in the computational sense, it still has an intended interpretation, and we can describe it as follows:

$$\text{queue}([f_1, \ldots, f_n], [b_1, \ldots, b_m]) = [f_1, \ldots, f_n, b_m, \ldots, b_1]$$

This just says exectly what we said above.

We can first give a definition for the list/1 function, which is intended to be the identity function on the interpretation of its argument, but needs to convert between the two representations of a finite sequence. The single clause is:

```
list(queue(F, B)) = append(F, reverse(B)).
```

This code uses the append/2 and reverse/1 functions that are defined in the list module of the standard library. Note that, here and elsewhere, we tend to use short variable names in order to keep the presentation compact in printed form. In real code it would be better to use more expressive names, such as `Front` and `RevBack`.

It is instructive to look at how the clause as a whole is interpreted. Since `F` and `B` are lists, and we have given the interpretations of the other symbols, plugging things in results in the following for each side of the equation:

$$\begin{aligned}
&\text{list}(\text{queue}([f_1, \ldots, f_n], [b_1, \ldots, b_m]) \\
&\quad = \text{list}([f_1, \ldots, f_n, b_m, \ldots, b_1]) \\
&\quad = [f_1, \ldots, f_n, b_m, \ldots, b_1] \\
&\text{append}([f_1, \ldots, f_n], \text{reverse}([b_1, \ldots, b_m])) \\
&\quad = \text{append}([f_1, \ldots, f_n], [b_m, \ldots, b_1]) \\
&\quad = [f_1, \ldots, f_n, b_m, \ldots, b_1]
\end{aligned}$$

Both sides are interpreted equally, so we can say that this clause is sound. Since this is a function with the default mode, the clause completeness condition is checked for us, and thus we can conclude that our implementation of list/1 is partially correct.

A simple version of eq/2 can be defined by converting both queues to lists:

```
eq(Q1, Q2) :- list(Q1) = list(Q2).
```

The clause soundness condition can easily be verified, since the symbols all denote identities. The clause completeness condition holds because the calls to list/1 cover every possible queue.

Implementing init/0 is also simple, as the empty queue must have both the front and back lists empty:

```
init = queue([], []).
```

We can define the four main operations as follows:

```
put(Z, queue(F, B), queue(F, [Z | B])).
get(Z, queue([Z | F], B), queue(F, B)).
unput(Z, queue(F, [Z | B]), queue(F, B)).
unget(Z, queue(F, B), queue([Z | F], B)).
```

As before, by plugging in our interpretations we can make judgements about correctness. Doing so, and then renaming lists of mixed *f*s and *b*s to lists of *t*s for clarity, we get Figure 2.6, which shows that our clauses match their intended interpretations exactly, which means the clause soundness condition is satisfied.

At this point we could compile the above predicates successfully. If we run a test program, however, calls to the `semidet` predicates will sometimes fail when there is supposed to be a solution. For example, putting an element in an empty queue and then trying to get it out again does not produce the element. The program has a missing answer bug!

The problem is that we have only checked the clause soundness condition, and not the clause completeness condition. For the `det` predicates put/3 and unget/3, the compiler checks this latter condition for us, but for the `semidet` predicates get/3

```
put(Z, queue(F, B), queue(F, [Z | B])).
```

$$\begin{aligned}&\text{put}(z, \text{queue}([f_1, \ldots, f_n], [b_1, \ldots, b_m]), \text{queue}([f_1, \ldots, f_n], [z, b_1, \ldots, b_m]))\\&\quad \Leftrightarrow \text{put}(z, [f_1, \ldots, f_n, b_m, \ldots, b_1], [f_1, \ldots, f_n, b_m, \ldots, b_1, z])\\&\quad \Leftrightarrow \text{put}(z, [x_1, \ldots, x_{m+n}], [x_1, \ldots, x_{m+n}, z])\end{aligned}$$

```
get(Z, queue([Z | F], B), queue(F, B)).
```

$$\begin{aligned}&\text{get}(z, \text{queue}([z, f_1, \ldots, f_n], [b_1, \ldots, b_m]), \text{queue}([f_1, \ldots, f_n], [b_1, \ldots, b_m]))\\&\quad \Leftrightarrow \text{get}(z, [z, f_1, \ldots, f_n, b_m, \ldots, b_1], [f_1, \ldots, f_n, b_m, \ldots, b_1])\\&\quad \Leftrightarrow \text{get}(z, [z, x_1, \ldots, x_{m+n}], [x_1, \ldots, x_{m+n}])\end{aligned}$$

```
unput(Z, queue(F, [Z | B]), queue(F, B)).
```

$$\begin{aligned}&\text{unput}(z, \text{queue}([f_1, \ldots, f_n], [z, b_1, \ldots, b_m]), \text{queue}([f_1, \ldots, f_n], [b_1, \ldots, b_m]))\\&\quad \Leftrightarrow \text{unput}(z, [f_1, \ldots, f_n, b_m, \ldots, b_1, z], [f_1, \ldots, f_n, b_m, \ldots, b_1])\\&\quad \Leftrightarrow \text{unput}(z, [x_1, \ldots, x_{m+n}, z], [x_1, \ldots, x_{m+n}])\end{aligned}$$

```
unget(Z, queue(F, B), queue([Z | F], B)).
```

$$\begin{aligned}&\text{unget}(z, \text{queue}([f_1, \ldots, f_n], [b_1, \ldots, b_m]), \text{queue}([z, f_1, \ldots, f_n], [b_1, \ldots, b_m]))\\&\quad \Leftrightarrow \text{unget}(z, [f_1, \ldots, f_n, b_m, \ldots, b_1], [z, f_1, \ldots, f_n, b_m, \ldots, b_1])\\&\quad \Leftrightarrow \text{unget}(z, [x_1, \ldots, x_{m+n}], [z, x_1, \ldots, x_{m+n}])\end{aligned}$$

Figure 2.6: Code and semantics of initial versions of queue operations.

and unput/3, we need to make sure that all possible values of the second argument—which is the input argument of both predicates—are handled.

The second argument in the above clause for get/3 is `queue([Z | F], B)`, which only covers cases where the front list is not empty. We need to determine what should happen if the front list is empty.

Consider the following equivalence:

$$\text{queue}([], b) = \text{queue}(\text{reverse}(b), [])$$

This can easily be seen to be true by plugging in the interpretation of the various symbols. If, for the empty list case, we replace the left hand side of this equation with the right hand side *before* getting the element from the front of the list, the call will not fail, unless the entire queue is empty, in which case failure is the expected result. The code that performs this is as follows:

```
get(Z, queue([], B), queue(F, [])) :-
    [Z | F] = reverse(B).
```

To check the result, we can once again plug in the interpretations. The clause body gives us:

$$\begin{aligned}&[z, f_1, \ldots, f_n] = \text{reverse}([b_1, \ldots, b_m])\\ &\quad \Leftrightarrow [z, f_1, \ldots, f_n] = [b_m, \ldots, b_1]\end{aligned}$$

This is saying that the body is only true when this last equation holds. For the clause head we get:

$$\begin{aligned}&\text{get}(z, \text{queue}([], [b_1, \ldots, b_m]), \text{queue}([f_1, \ldots, f_n], []))\\ &\quad \Leftrightarrow \text{get}(z, [b_m, \ldots, b_1], [f_1, \ldots, f_n])\end{aligned}$$

If we use the equation derived from the clause body to substitute variables in the above, and then rename some variables, we end up with:

$$\begin{aligned}&\Leftrightarrow \text{get}(z, [z, f_1, \ldots, f_n], [f_1, \ldots, f_n])\\ &\Leftrightarrow \text{get}(z, [z, x_1, \ldots, x_n], [x_1, \ldots, x_n])\end{aligned}$$

This matches the intended interpretation, as required, and as the two clauses together now cover all possible solutions, we can conclude that the implementation is correct.

An extra clause for unput/3 can be derived in a similar way to the extra clause for get/3, except that here it is the front list that is reversed and put at the back:

```
unput(Z, q(F, []), q([], B)) :-
    [Z | B] = reverse(F).
```

Verifying the correctness of unput/3 is left as an exercise for the reader.

2.7 Declarative debugging

Declarative debugging is a debugging technique in which the debugger compares the actual solutions of calls (obtained by running the program) with their intended solutions in the declarative semantics (obtained by asking the programmer).

The process of debugging, broadly speaking, can be broken down into the following three phases.

1. Observe a bug symptom. In the classical semantics, the two types of symptoms that the programmer can observe are wrong answers and missing answers.

2. Localize the bug. The programmer narrows the problem down to a part of the source, preferably small, that is able to explain at least some of the incorrect behaviour.

3. Fix the bug. The programmer reasons about the code, in order to try to determine how to make it correct.

Declarative debugging applies to the second phase. The technique is an algorithm, which may be followed manually by the programmer or may be automated by a debugging tool, that is able to narrow the immediate source of a bug down to a single predicate or function. Theorem 1 implies that this is possible.

```
:- type digits == list(int).

:- func to_string(digits) = string.
to_string([]) = "0".
to_string(As @ [_ | _]) =
    append_list(map(string, reverse(As))).

:- func add(digits, digits) = digits.
add([], Bs) = Bs.
add(As @ [_ | _], []) = As.
add([A | As], [B | Bs]) = [A + B | add(As, Bs)].
```

Figure 2.7: A buggy implementation of `digits`, which represent arbitrary precision integers.

Consider the snippet of (buggy) code in Figure 2.7, that represents arbitrary precision integers as lists of decimal digits starting from the least significant. (This means that e.g. the list `[5,1]` represents the integer 15.) It provides a function to convert an integer to a string, and a function to add two integers.

If we call to_string(add([2], [5, 1])) the answer "17" is returned, which is the answer that we intended (since $2 + 15 = 17$). If we call to_string(add([6], [7])) it returns "13", which is also the answer that we intended (since $6 + 7 = 13$). In this case there were incorrect intermediate values, but we did not observe this in the result so we are not in a position to commence debugging.

If we call to_string(add([2, 6, 1], [3, 7])) then we intend it to return "235" (since $162 + 73 = 235$), but instead it returns "1135", which is a wrong answer. Thus we have observed a bug symptom.

To localize this bug we start at the bug symptom, which for us is the atom that was incorrect:

```
to_string(add([2,6,1], [3,7])) = "1135"
```

We can first check the call to add/2. In this case the following atom appears:

```
add([2,6,1], [3,7]) = [5,13,1]
```

This is false in the intended interpretation, because integers are supposed to be represented by a list whose elements are all in the range 0 to 9, and 13 is outside

that range. Since this is a wrong answer, we proceed to debug the atom. It matches the third clause of add/2, the instance of which contains the following calls:

```
2 + 3 = 5
add([6,1], [7]) = [13,1]
```

The first is obviously true, but the second is false because $16 + 7 = 23$, so we intended the result to be $[3, 2]$. Again, the third clause is matched and the calls made are:

```
6 + 7 = 13
add([1], []) = [1]
```

In this case *both* atoms are true in the intended interpretation. Since this instance of the third clause of add/2 takes the results of these calls, all of which are correct, and constructs an incorrect result from them, we can conclude that the third clause contains a bug, And indeed it does, since it does not allow for a carry bit to flow over to the next digit.

Figure 2.8 shows the transcript of an `mdb` session, that uses the declarative debugger to find this problem. The process looks a bit different to how we just described it, but that is because the debugger assumes that builtin functions such as + are correct, which means that it does not ask questions about their correctness.

Note `mdb` uses "contour" to mean "a path execution takes through the body of a clause". In cases like this where the clause is a simple conjunction, this means all the goals in the clause, but in clauses containing if-then-else or disjunctions, it would mean just the goals along the path execution took inside those constructs to compute the wrong answer being debugged.

Bug fixing, the third phase of debugging, might start with defining a function add_with_carry/3, where the intended interpretation is:

$$\text{for } a, b \in [0..9],\ c \in \{0, 1\}, \text{add_with_carry}(a, b, c) = a + b + c$$

Then add/2 could be implemented as:

```
add(As, Bs) = add_with_carry(As, Bs, 0).
```

Implementing add_with_carry/3 is left as an exercise for the reader.

2.8 Summary

Interpretations of function and predicate symbols provide us with a meaning for our programs. Herbrand interpretations give the meaning from the compiler's point of view, in purely syntactic terms. More generally, interpretations describe, sometimes informally, the behaviour of the functions and predicates in a program with regard to its inputs and outputs.

The interpretation of multi-moded predicates greatly helps in clarifying how different modes should behave. This is particularly important when considering

predicates and functions that use mode-dependent clauses (such as the predicate append/3 in the string module of the standard library) whose consistency the compiler cannot verify in the general case.

One interpretation in particular is called the intended interpretation. This acts as a specification that reflects what the programmer intends to implement. With it, we can determine the partial correctness of a program with respect to wrong answer and missing answer bugs.

In particular, it is expressive enough to allow us to determine whether an individual clause is sound, and whether a definition is complete. Thus, it can assist with bug localization and fixing, as well as help with avoiding bugs in the first place. We have given some simple examples to illustrate this, as well as introducing a queue ADT which we use as a running example.

In the remainder of this guide we take a more formal look at how Mercury's semantics is defined, as well as some additional topics of interest.

```
Melbourne Mercury Debugger, mdb version DEV.
Copyright 1998-2012 The University of Melbourne.
Copyright 2013-2023 The Mercury team.
mdb is free software; there is absolutely no warranty...
       1:      1  1 CALL pred arbint.main/2-0 (det)
mdb>
       2:      2  2 CALL func arbint.add/2-0 (det)
mdb> f
      17:      2  2 EXIT func arbint.add/2-0 (det)
mdb> dd
add([2, 6, 1], [3, 7]) = [5, 13, 1]
Valid? n
add([6, 1], [7]) = [13, 1]
Valid? n
add([1], []) = [1]
Valid? y
Found incorrect contour:
+(6, 7) = 13
add([1], []) = [1]
add([6, 1], [7]) = [13, 1]
Is this a bug? y
      16:      4  3 EXIT func arbint.add/2-0 (det)
mdb> quit -y
```

Figure 2.8: An mdb declarative debugging session.

Chapter 3

First-order predicate calculus

3.1 Overview

In this chapter we introduce first-order predicate calculus, also known as first-order logic, or classical logic. This mathematical notion forms the basis of Mercury's declarative semantics, via the translation of Mercury code into axioms of a predicate calculus theory.

Articles on first-order predicate calculus typically define a syntax, then introduce a deductive system that allows proofs to be constructed from inference rules, then define a semantics in terms of "interpretations" and "models" and prove some results in the meta-theory. We will take a similar approach, however we present the semantics, with a particular focus on how (first-order) Mercury programs are converted to logic formulas, *before* we cover the deductive system. In the next chapter we will present the deductive system, which gives a computational interpretation to the logic.

Mercury supports higher-order code, so a first-order description will not directly cover all of it. First-order logic is also untyped. Despite these differences, we can reduce Mercury's declarative semantics to first-order logic, which is conceptually much simpler than the alternatives—first-order theories have relatively simple models, such as the Herbrand interpretations from the previous chapter—so that is the approach we will take in this guide. We will discuss types and higher-order code, plus some other extensions of the basic logic, in Chapter 6.

In the remainder of this chapter we give the syntax of predicate calculus, and show how basic Mercury constructs generate formulas, and how Mercury programs generate axioms. We then give a formal definition of the classical semantics that results from those axioms. Some examples of ad hoc proofs based on the semantics are provided, to help illustrate the concepts introduced, and to motivate the operational semantics that we define in detail in the next chapter. We conclude the chapter with some philosophical remarks about the role of classical logic in understanding computer programs.

1. An infinite set of variables: a, b, x, y, z, ...
2. Logical constants: *true*, *false*
3. Logical connectives: $\wedge$, $\vee$, $\neg$, $\leftarrow$, $\rightarrow$, $\leftrightarrow$, ...
4. Quantifiers: $\forall$, $\exists$
5. Logical relations: $=$
6. Punctuation: we will use parentheses and commas in the usual way.
7. A set of function symbols f, g, ..., each of which has a non-negative arity, which we denote as, for example, f/2, g/1, ...
8. A set of predicate symbols p, q, ..., each of which has a non-negative arity, which we denote as, for example, p/2, q/3, ...

Figure 3.1: Symbols used in predicate calculus.

3.2 Syntax

The syntax of first-order logic is given in terms of a set of symbols, along with rules to say how they can be put together to make well-formed terms and formulas. The symbols we use are listed in Figure 3.1.

Some sources refer to the symbols in categories 1 to 6 as *logical*, while referring to the symbols in categories 7 and 8 as *non-logical*. It is important to note that this term is being used in a different sense to that in Section 2.4. In this context it is referring to whether symbols are defined as part of the logic itself, or defined as part of a particular theory. To put it in programming terms, "logical" here means that the symbol is a fixed part of the language, while "non-logical" really just means user-defined.

By convention, we will use italics such as a, b, x, y and z, possibly with subscripts, to stand for arbitrary variables. A sequence of variables may be written with an overbar, as in $\bar{x}$. The logical constants, logical connectives and quantifiers have their usual meanings, and we will write terms and formulas using the following rules of operator precedence:

- The equals operator, '$=$', binds more tightly than any logical connective.
- The negation operator, '$\neg$', binds more tightly than any of the other logical connectives.
- Conjunction, '$\wedge$', binds next most tightly after negation.
- Disjunction, '$\vee$, binds next most tightly after conjunction.

Terms:

t	$::=$	x	variable
	$\mid$	a	constant
	$\mid$	$\mathrm{f}(t_1, \ldots, t_n)$	compound term

Formulas:

ϕ	$::=$	*true*	always true
	$\mid$	*false*	always false
	$\mid$	$t_1 = t_2$	equation
	$\mid$	$\mathrm{p}(t_1, \ldots, t_n)$	predicate call
	$\mid$	$\phi_1 \wedge \ldots \wedge \phi_n$	conjunction
	$\mid$	$\phi_1 \vee \ldots \vee \phi_n$	disjunction
	$\mid$	$\neg\phi_1$	negation
	$\mid$	$\forall x.\, \phi_1$	universal quantification
	$\mid$	$\exists x.\, \phi_1$	existential quantification

Figure 3.2: The grammar rules for terms and formulas.

- The remaining operators, for implication and bi-implication, bind the least tightly.

We use parentheses to override these rules in the usual manner; parentheses are always used for disambiguation in the case of implications and bi-implications. The scope of a quantifier extends as far to the right as possible; parentheses will be used if the scope needs to be limited.

The user-defined symbols in the semantics of a Mercury program come from the declarations in that program. Function symbols in the semantics include data constructors declared in discriminated union types, as well as declared functions. Predicate symbols come from declared predicates. We will use names such as f and g to stand for arbitrary function symbols, and names such as p and q to stand for arbitrary predicate symbols. We will sometimes refer to function symbols with arity zero as *constants*, and use names such as c and d to stand for arbitrary constants.

The arity of a predicate or function symbol is listed after a slash, for example f/2 or p/1. However, we will sometimes leave the arity off entirely when it is zero, and/or when the arity is clear from the context. Note that the arity is part of each symbol's identity: two symbols with the same name but different arities are considered different symbols.

The grammar rules for the predicate calculus are shown in Figure 3.2. Terms are constructed from variables and function symbols in essentially the same way that expressions are in Mercury. We will refer to arbitrary terms using names such as s and t. Terms that are constructed using only variables and data constructors (that is, with no function calls) play an important role in our discussion; we will refer to these as *data terms*. A data term that contains no variables we will refer to as a *ground data term*.

We will assume the existence of a constant for each integer, and the existence of two function symbols, $[|]/2$ and $[]/0$, representing the data constructors for nonempty lists and the empty list respectively. We will write list terms in an analogous way to Mercury list syntax, for example, we will write $[1, 2, 3]$ as a shorthand for $[|](1, [|](2, [|](3, [])))$.

Formulas are either atomic or compound. Atomic formulas (atoms) are either logical constants, equations, or predicate calls. Compound formulas are constructed from atoms using connectives and quantifiers, in essentially the same way that goals are in Mercury. We will refer to arbitrary formulas using names such as ϕ and ψ.

Quantifiers with multiple variables stand for the same quantifier with each variable in turn, that is, $\forall xy.\phi$ is an abbreviation for $\forall x.\forall y.\phi$. A variable occuring in a formula is *bound* if it is captured by a quantifier, otherwise it is *free*. For example, x is bound and y is free in the formula $\forall x.\, \mathrm{f}(x, y) = y$. (Despite the shared nomenclature, these notions are different from the notions of bound and free in Mercury's mode system.)

A formula that has no free variables is said to be *closed*; some sources refer to closed formulas as "sentences". The universal closure of a formula is that formula with all of its free variables universally quantified.

3.3 Expressions and goals

One of the building blocks for understanding Mercury's declarative semantics is to see how Mercury expressions are mapped to terms, and how Mercury goals are mapped to formulas. The relationship we want to describe here is the one represented by the lower left vertical arrow in Figure 1.1 on page 2.

The mappings for expressions and goals are provided in full detail in the reference manual, but for convenience Figure 3.3 summarizes the most important bits. In line with our notational convention, ϕ is the formula that corresponds to goals such as `Goal`. For goals that have more than one subgoal, they are distinguished by suffixes in the expressions, and by the same suffixes being used as subscripts in formulae, so that `Goal1` maps to ϕ_1, and so on.

Some kinds of goals the Mercury compiler implements by reducing to other kinds of goals. This needs to be done with some care, as operationally Mercury needs to be able to make sense of the resulting goals in terms of constructively binding variables. Goals in the last four rows of this table, unidirectional implications, bidirectional implications, and universal quantification, are handled in this way.

In classical logic, the semantics of unidirectional implication, in both its forward `Goal1 => Goal2` form and in its equivalent backward form `Goal2 <= Goal1`, is usually given as $(\neg\phi_1) \vee \phi_2$. This short formula is adequate for the purposes of classical logic, because, when given a ground instance of this goal, it can tell you whether it is true or false in a given interpretation. However, it is inadequate for a programming language, whose concern is not judging whether a ground instance

Expressions ⇒ Terms:

`X`	x
`c`	c
`f(t1, ..., tN)`	$\mathrm{f}(t_1, \ldots, t_n)$

Goals ⇒ Formulas:

`true`	*true*
`false`	*false*
`t1 = t2`	$t_1 = t_2$
`p(t1, ..., tN)`	$\mathrm{p}(t_1, \ldots, t_n)$
`not Goal`	$\neg\phi$
`Goal1, ..., GoalN`	$\phi_1 \wedge \ldots \wedge \phi_n$
`( Goal1 ; ... ; GoalN )`	$\phi_1 \vee \ldots \vee \phi_n$
`( if C then T else E )`	$(\phi_c \wedge \phi_t) \vee (\neg\phi_c \wedge \phi_e)$
`some [X] Goal`	$\exists x.\, \phi$
`all [X] Goal`	$\forall x.\, \phi$
`Goal1 => Goal2`	$\phi_1 \rightarrow \phi_2$
`Goal1 <= Goal2`	$\phi_1 \leftarrow \phi_2$
`Goal1 <=> Goal2`	$\phi_1 \leftrightarrow \phi_2$

where ϕ is the formula corresponding to `Goal`

Figure 3.3: Semantics of Mercury expressions and goals.

of a formula is true, but *finding* the true ground instances, or, in other words, *computing* the values of the variables that make the formula true. For this purpose, viewing `Goal1 => Goal2` as $(\neg\phi_1) \vee \phi_2$ is limiting, because the fact that ϕ_1 is negated does not allow variable bindings to flow from ϕ_1 to ϕ_2. Mercury therefore views the semantics of `Goal1 => Goal2` as being $(\neg(\phi_1 \wedge (\neg\phi_2))$. This formula *does* allow variable bindings to flow from ϕ_1 to ϕ_2.

The formulas $(\neg\phi_1) \vee \phi_2$ and $\neg(\phi_1 \wedge (\neg\phi_2))$ are equivalent in classical logic, which is concerned with truth, but they are *not* equivalent in *intuitionistic logic* (sometimes called *constructive logic*), which is concerned with computability as well as truth. The main difference between the two logics is that while both accept proofs of the form "there exists an entity with this given property, and here is how you construct that entity", only classical logic accepts proofs of the form "there must exist an entity with this given property, because assuming its nonexistence leads to a contradiction, but we don't have a method of constructing it". There are therefore propositions that classical logic would say are true for which intuitionistic logic cannot say whether they are true or false.

Just as $(\neg\phi_1) \vee \phi_2$ is the obvious formula for `Goal1 => Goal2`, $\forall x.\, \phi$ is the obvious formula for `all [X] Goal`. However, this formula does not work in a programming language either, because it does not provide a useful way to enumerate the values of `X` for which `Goal` should be checked to be true. (In theory, we could try to check whether `Goal` is true for all values in the type of `X`, but even types as simple as "list of integers" have an infinite number of values, so that does not work.) Mercury therefore views the semantics of `all [X] Goal` as being $\neg(\exists x.\, \neg\phi)$. This formula is equivalent to $\forall x.\, \phi$ in classical logic, but it allows code within ϕ itself to act the generator of the values of x. For example, in the frequent case where the goal inside the universal quantification is an implication, such as

```
all [Item] (
    set.member(Item, Set) => property(Item)
)
```

the goal calling `set.member` will return all the items in `Set` one at a time as Item (`Set` having been bound by earlier code), until either

- one of the items fails the property test (in which case the universal quantification goal fails), or
- there are no more items left to return (in which case the universal quantification goal succeeds).

In cases where the goal inside the universal quantification is a bidirectional implication, both subgoals of that implication must be able to generate values for the quantified variable. We will discuss the semantics of bidirectional implications at the end of the next section.

3.4 Implicit quantification

The usual rule in mathematics is that free variables in a formula are implicitly universally quantified across the whole formula. (For example, in a mathematical identity such as $\sin^2\theta + \cos^2\theta = 1$, the equation is meant to be taken as true for *all* values of θ.) This is not quite what we want for Mercury's semantics, because in Mercury code there are common situations in which universal quantification over the entire clause clashes with what most programmers expect. To avoid violating the "law of least astonishment", Mercury therefore quantifies some variables differently to the usual mathematical rule.

In first-order code, the most important exception consists of variables that are intended to be bound by the condition of an if-then-else. The values of such variables are available in the rest of the condition (the part after the goal that bound them) and in the then-part of the if-then-else, but

- they are not available in the else-part of that if-then-else because that code is executed only if the condition fails, and failed computations cannot generate any bindings; and
- they are not available in any part of the clause outside of that if-then-else, because execution may get there after taking the else branch in that if-then-else, which again means that the variable binding won't have taken place.

This means that variables that the programmer intends to be bound by the condition of an if-then-else may occur in that condition and in the then-part that follows it, but nowhere else in the clause. Mercury implicitly quantifies an if-then-else `( if C then T else E )` where $\bar{x}$ is the set of the variable for which this is true like this:

$$(\exists \bar{x}.\, \phi_c \wedge \phi_t) \vee (\neg(\exists \bar{x}.\, \phi_c) \wedge \phi_e)$$

Note that

- the variables in $\bar{x}$ are quantified existentially and in these scopes derived from just the if-then-else goal, rather than universally and over the scope of the entire clause, and
- the first quantifier's scope includes the then-branch goal but the second quantifier's scope does not include the else-branch goal, reflecting the fact that such variables cannot be used there.

This implicit introduction of existential quantifiers interacts with the semantics of bidirectional implications (or bi-implications for short). The obvious semantics of `Goal1 <=> Goal2` is the conjunction of the formulas for `Goal1 => Goal2` and `Goal2 => Goal1`, which for Mercury is

$$(\neg(\phi_1 \wedge (\neg\phi_2))) \wedge (\neg(\phi_2 \wedge (\neg\phi_1)))$$

Clauses ⇒ Formulas:

`p(t1, ..., tN) :- Body`	$\mathrm{p}(t_1, \ldots, t_n) \leftarrow \phi$
`p(t1, ..., tN)`	$\mathrm{p}(t_1, \ldots, t_n) \leftarrow \mathit{true}$
`f(t1, ..., tN) = t :- Body`	$\mathrm{f}(t_1, \ldots, t_n) = t \leftarrow \phi$
`f(t1, ..., tN) = t`	$\mathrm{f}(t_1, \ldots, t_n) = t \leftarrow \mathit{true}$

where ϕ is the formula corresponding to `Body`, if present

Figure 3.4: Mercury clauses as reverse implicatios.

However, this does have the effect of duplicating both ϕ_1 and ϕ_2. If either ϕ_1 or ϕ_2 (or both) contain if-then-elses, then any variable that *used* to occur only in the condition and the then-part of such an if-then-else, will now occur outside of that if-then-else: specifically, it will occur in the other copy of that if-then-else. This is why the Mercury compiler expands out bi-implications (in other words, it replaces `Goal1 <=> Goal2` with the conjunction of `Goal1 => Goal2` and `Goal2 => Goal1`) only *after* it has inserted all needed existential quantifiers into if-then-elses. Bi-implications are the only Mercury constructs whose implementation in terms of other Mercury constructs can duplicate subgoals, so other Mercury constructs that are implemented this way (such as unidirectional implications) are expanded out much earlier.

3.5 Clauses

To give a semantics to clauses, we can consider a mapping that just replaces `:-` with reverse implication. Figure 3.4 shows the effect this has on the different forms of clauses—the formulas that result are implications in which the body implies the head, which is the interpretation discussed in Section 2.5. This essentially lines up with clause soundness, in that if one of these implications is incorrect it will lead to a wrong answer bug.

Two problems still remain with this formulation of clauses. First, the resulting formulas are not necessarily closed. We will see shortly that we require the formulas that go into our semantics to be closed. Second, while we have made the connection to clause soundness, we have not yet done so for clause completeness. That will require looking at all clauses that define a predicate or function, instead of just one at a time.

Before we continue our discussion of clauses, however, we will first need to take a closer look at equality as it relates to the predicate calculus. While the topic is usually taken as basic in logic, there are slightly different approaches and it is worth looking closely at the choices we have made.

3.6 First-order logic with equality

In our syntax we included equality as one of the logical symbols. We define this as *semantic* equality: two terms are equal if and only if they denote the same thing. Thus, by definition, the relation is reflexive, symmetric, and transitive, as would be expected. A substitutivity principle also applies, in that if any argument to a function is replaced by another argument that is equal according to our definition, then the result is equal. Similarly, if any argument to a predicate is replaced by another equal argument, then the result is logically equivalent.

Another possible definition of equality, which we do not adopt, is *syntactic* equality. This means that two terms are considered equal if they are syntactically identical, or if they can be made so via a variable substitution. The two definitions are very similar, and produce the same results in most cases. There are subtle differences, however, and these will eventually become relevant when we talk about `semidet` functions in Section 6.2, so it important to take note of precisely which definition we are using.

When the equality relation is defined as part of the logic, as we have done in this guide, the logic is sometimes called *first-order logic with equality*. Some authors take an alternative approach where the equality relation is treated no differently to other predicate symbols. This alternative approach is sometimes used in order to define equality in terms of some other relation, for example by saying that two sets are equal if each is a subset of the other. In typical cases, however, equality is taken to be part of the logic, and the term "first-order logic" is usually taken to mean first-order logic with equality.

We have chosen to include equality as part of the logic since it is simpler to do so, and we do intend the aforementioned properties to hold. This does not capture our full intent, however, and we will need to add to our definition in the next section.

3.7 Axioms

3.7.1 What are axioms?

Axioms are closed formulas that are taken to be true without proof, meaning they may be used as a starting point for proofs of other formulas. We will look more at proofs in the next chapter. A closed formula that can be proven from a set of axioms is called a *theorem*, and the collection of all such closed formulas is known as a *theory*. Thus, the axioms form the basis of a particular predicate calculus theory.

In this section we will show how to generate a set of axioms from the declarations and clauses in a Mercury program. The theory that results will ultimately determine the declarative semantics of the program. We will make this concept clearer once we have discussed models, which we will do in Section 3.8.

Note that in some sources on this topic the authors' aim is to axiomatize the logic itself—that is, include axioms that define the logical symbols—but we take

the approach of defining the logic symbols directly and just using axioms to define what is specific to a program. This is because we want to state something *using* the logic, rather than explore theorems *about* the logic. Thus, when we refer to "axioms" we mean those that are generated from the program, rather than the sort that define the logic being used.

3.7.2 Equality axioms

Our definition of equality thus far gives us some conditions that imply when two given terms are equal. It does not, however, say when they are not equal. As it stands, we are not even able to rule out the case where *all* terms are equal. To repair this, we will need to add some axioms relating to equality. Our intent is that two ground data terms (that is, terms that do not include function calls) should be equal only if they are syntactically identical.

Note that we are saying *only if*. In other words, equality of ground data terms implies that they must be syntactically identical. This is not the same as saying syntactic identity of ground data terms implies that they must be equal, since the implication is in the opposite direction. If the implication were this way around we would be defining equality as syntactic, but, as mentioned in Section 3.6, that is not the definition of equality we have adopted.

The condition that ground terms are equal only if identical is known as the *Unique Names Assumption*. As the name suggests, this is often left implicit, particularly when discussing logic programming or databases. In other cases, such as when dealing with ontologies, it can be desirable to have two distinct names denote the same thing, in which case the assumption is not made. In our case, there are of course terms containing function calls that are intended to be equal even if they are not syntactically identical. For example $2 + 2$ is equal to $1 + 3$ since both sides are equal to 4, so we do not want this assumption to always apply. We do intend that this assumption holds for ground data terms, however.

Another assumption we wish to make is that all terms are finite. For an equation such as $x = \mathrm{f}(x)$, where f is a data constructor, the solution if it exists must be the infinite term:

$$x = \mathrm{f}(\mathrm{f}(\mathrm{f}(\mathrm{f}(\ldots))))$$

This is not a term that can be written down using our syntax, however, so we wish to exclude it as a possible solution.

To support these requirements, a number of axioms are generated for each type. These are known as the *free equality axioms*. Consider the type definitions in Figure 3.5. The constants are a, b and nil, and cons/2 is the only other function symbol. Note that foo and bar are not included, as these are type constructors not data constructors or declared functions.

The free equality axioms for foo and bar are shown in Figure 3.6. The two in the first row say that ground data terms are not equal unless their principal functors are equal. We have omitted any axioms that say terms of different types are not equal,

```
:- type foo ---> a ; b.
:- type bar ---> nil ; cons(foo, bar).
```

Figure 3.5: Type definitions for the foo and bar types.

$$\neg(\mathrm{a} = \mathrm{b}) \qquad \forall xy.\, \neg(\mathrm{nil} = \mathrm{cons}(x, y))$$

$$\forall x_1 x_2 y_1 y_2.\, (\mathrm{cons}(x_1, y_1) = \mathrm{cons}(x_2, y_2)) \rightarrow (x_1 = x_2 \wedge y_1 = y_2)$$

$$\forall x.\, \neg(x = \mathrm{cons}(t_1, t_2)), \text{ if } x \text{ occurs in } t_1 \text{ or } t_2$$

Figure 3.6: Equality axioms for the foo and bar types.

since in general there is a large number of these, and in a type correct program such axioms will not make any real difference. In a non-typechecked setting, all $O(n^2)$ axioms would be required.

The third axiom says that ground data terms are not equal unless their arguments are equal. In other words, this simply states that cons/2 denotes an injective function.

The last line is an axiom schema (that is, an infinite family of axioms), where there is one axiom for each pair of data terms t_1 and t_2. This schema is known as the *occurs check*,[1] and it states that a variable is never equal to a data term that contains that variable. Equivalently, a data term never strictly contains itself as a subterm. This implies that all data terms are finite; for example the formula $x = \mathrm{cons}(\mathrm{a}, x)$, which would otherwise describe an infinite term, is always false because x occurs in the term that it is supposedly equal to.

With the free equality axioms we can no longer say that ground data terms that are syntactically different may be equal. For example, the axioms allow us to infer

$$\neg(\mathrm{cons}(\mathrm{a}, \mathrm{nil}) = \mathrm{cons}(\mathrm{b}, \mathrm{nil}))$$

since if we assume that cons(a, nil) and cons(b, nil) are equal, then by the third axiom we have that a = b. This contradicts the first axiom, which means that our assumption must have been false.

Without the free equality axioms there would have been no way to derive this proof.

3.7.3 Clause soundness axioms

In Section 2.5 we gave the clause soundness condition for partial correctness, which required that each clause in the program must be true in the intended interpretation. This can be expressed as an axiom in a straightforward way.

[1] Or sometimes "occur check", without the plural.

Recall from Section 3.5 that a clause for a predicate p/n can be mapped to a formula $\psi \leftarrow \phi$, where ψ corresponds to the clause head and takes the form $\mathrm{p}(t_1, \ldots, t_n)$, for argument terms $t_1, \ldots, t_n$, and ϕ is the formula corresponding to the clause body (or *true* if the clause is a fact). Similarly for a function f/n, except that ψ takes the form $\mathrm{f}(t_1, \ldots, t_n) = t_r$, where t_r is the return expression.

The formula can be made into an axiom in the same way mathematical formulas usually are, by taking the universal closure as follows:

$$\forall \bar{x}.\, \psi \leftarrow \phi$$

where $\bar{x}$ is the set of free variables that occur in ψ and ϕ.

The resulting formula is known as the *clause soundness axiom* for the clause. As suggested above, it expresses the clause soundness condition for the clause, and it must be true in the intended interpretation. Since it is an implication, the only way it can be false is if ψ is true and ϕ is false. If this is the case—that is, the axiom is false in the intended interpretation—then, in line with the discussion in Section 2.5, the clause is a wrong answer bug.

Observe that, while for predicates the axiom asserts something about ground atoms for the predicate itself, for functions the axiom asserts something about the equality relation. Unlike data constructors, we do not generate axioms that say the function is injective, or that the function returns values that are different from every other function. The function completion adds new instances of terms being equal, rather than excluding such instances.

The axiom also tells us a way to make inferences about the program. We can choose any way we like of assigning values to the variables $\bar{x}$, and if we do so in a way that ϕ, the clause body, is something we have already established to be true, then we can infer that ψ, the clause head, will also be true. Equivalently, if we want to know whether or not an instance of ψ is true, it is sufficient to show that ϕ is true under the same variable assignment.

3.7.4 Combined clause soundness axioms

In the form given in the previous section, the clause soundness axiom is convenient for reasoning about a program one clause at a time. It is also useful, however, to be able to reason about the combination of all clauses that make up a predicate or function definition.

One way to express a combined clause soundness axiom is to directly conjoin all of the individual axioms, as follows:

$$\forall \bar{x}_1.\, \psi_1 \leftarrow \phi_1 \wedge \ldots \wedge \forall \bar{x}_m.\, \psi_m \leftarrow \phi_m$$

where m is the number of clauses in the definition. This does not provide us with much additional insight as it stands, since there are still just as many implications to consider. If, however, the clause heads all have identical arguments, we could easily combine all the conjuncts into a single implication.

We can make the clause heads take the required form by performing a simple logical transformation. For a predicate p/n, let $v_1, \ldots, v_n$ be a sequence of variables that are distinct from any other variables in the clauses. We define the following formulas:

$$\begin{aligned} \psi' &\equiv \mathrm{p}(v_1, \ldots, v_n) \\ \phi' &\equiv \exists \bar{x}.\, v_1 = t_1 \wedge \ldots \wedge v_n = t_n \wedge \phi \end{aligned}$$

where $\bar{x}$ is the set of free variables that occur in ψ and ϕ, and $t_1, \ldots, t_n$ are the original argument terms. In other words, ψ' is ψ with the fresh variables in place of the argument terms, and ϕ' is ϕ conjoined with equations between each of the fresh variables and the corresponding argument terms, and with all variables except the fresh ones existentially quantified.

Analogous definitions can be given for functions, with the difference being that we also need a variable, v_r, for the function result, and an additional equation to bind it. In this case we obtain:

$$\begin{aligned} \psi' &\equiv \mathrm{f}(v_1, \ldots, v_n) = v_r \\ \phi' &\equiv \exists \bar{x}.\, v_1 = t_1 \wedge \ldots \wedge v_n = t_n \wedge v_r = t_r \wedge \phi \end{aligned}$$

where $\bar{x}$ and $t_1, \ldots, t_n$ are as before, and t_r is the return expression for the clause.

Now consider the formula $\psi' \leftarrow \phi'$. If we take the universal closure in the same way as in the previous section, the result is logically equivalent to the clause soundness axiom. Intuitively, moving argument terms into the body via equations with fresh variables does not change the meaning of a clause. The reason we existentially quantify variables in the original clause is because we are only interested in the values of head variables, and because when we combine the clauses, as we will do in a moment, we do not want variables from different clauses to clash if they happen to have the same name. Essentially, the existential quantification reflects that the scope of variables in a clause is just that single clause.

From the collection of clauses in a definition, we obtain ψ', which is common to all the clauses, and which is implied by ϕ'_i for the ith clause. The conjunction of the formulas is found by taking the disjunction of the antecedents:

$$\psi' \leftarrow \phi'_1 \vee \ldots \vee \phi'_m$$

Universally quantifying the free variables, $\bar{v}$, would give us something equivalent to the conjunction of all of the clause soundness axioms for the definition.

3.7.5 Clause completeness axioms

Our definition of partial correctness from Section 2.5 requires not just clause soundness but also clause completeness. That is, the set of clauses defining a predicate or function must, between them, cover every possible ground atom that is true in the intended interpretation.

The *clause completeness axiom*, which expresses the clause completeness condition, is the counterpart to the clause soundness axioms. It can be obtained from the combined clause soundness axiom from the previous section, by changing the reverse implication into a forward implication prior to universally quantifying.

For a predicate or function defined by m clauses, the clause completeness axiom is therefore:

$$\forall \bar{v}.\, \psi' \rightarrow \phi'_1 \vee \ldots \vee \phi'_m$$

Like the clause soundness axioms, it must be true in the intended interpretation. In this case, the only way it can be false is if the left-hand side is true and the right-hand side is false. In line with the discussion in Section 2.5, if this happens then the definition contains a missing answer bug.

Intuitively, putting the clauses into a disjunction reflects the fact that execution can choose any one of the clauses, and using a forward implication means that, while each clause says what things are true, these are the *only* things that are true. That is, every ground atom that is true must be covered by one of the clauses, as we expect.

We can use the clause completeness axiom to make inferences about the program, in much the same way as we can with the clause soundness axiom. The difference is that it gives us *negative* information—it allows us to infer that a particular ground atom must be false, rather than true. This in turn allows us to reason about whether the negation of a ground atom is true, or about which branch will be taken by a conditional goal.

3.7.6 Predicate and function completion

In most sources, the clause soundness axioms and the clause compleness axiom are combined into a single formula. For a given predicate or function, this formula, which is logiclly equivalent to the conjunction of the axioms, is known as the *completion* of the predicate or function.

Since we already have the axioms in the form of implications in one direction or the other, the conjunction of them is just a bi-implication between the two sides. In other words, the formula is:

$$\forall \bar{v}.\, \psi' \leftrightarrow \phi'_1 \vee \ldots \vee \phi'_m$$

where $\bar{v}$ is the set of fresh variables we introduced.

An example should help to illustrate, and for this we will turn once again to append/3. The clauses, when mapped to the $\psi \leftarrow \phi$ form, are as follows (variable names are chosen to fit with our notational convention):

$$\text{append}([], y, y) \leftarrow \mathit{true}$$
$$\text{append}([w|x], y, [w|z]) \leftarrow \text{append}(x, y, z)$$

We obtain the clause soundness axioms by universally quantifying:

$$\forall y.\ \text{append}([], y, y) \leftarrow \mathit{true}$$
$$\forall wxyz.\ \text{append}([w|x], y, [w|z]) \leftarrow \text{append}(x, y, z)$$

To get the combined formula, we introduce fresh variables $v_1, \ldots, v_3$ and make the following definitions:

$$\begin{aligned}
\psi' &\equiv \text{append}(v_1, v_2, v_3) \\
\phi'_1 &\equiv \exists y.\, v_1 = [] \wedge v_2 = y \wedge v_3 = y \\
\phi'_2 &\equiv \exists wxyz.\, v_1 = [w|x] \wedge v_2 = y \wedge v_3 = [w|z] \wedge \text{append}(x, y, z)
\end{aligned}$$

Note that we have omitted *true* from the conjunctions, since they do not have any effect. Putting the definitions together we get the following clause completeness axiom:

$$\begin{aligned}
&\forall v_1 v_2 v_3.\ \text{append}(v_1, v_2, v_3) \rightarrow \\
&\quad (\exists y.\, v_1 = [] \wedge v_2 = y \wedge v_3 = y) \vee \\
&\quad (\exists wxyz.\, v_1 = [w|x] \wedge v_2 = y \wedge v_3 = [w|y] \wedge \text{append}(x, y, z))
\end{aligned}$$

The completion of append/3 is the same as the clause completeness axiom, except it uses a bi-implication:

$$\begin{aligned}
&\forall v_1 v_2 v_3.\ \text{append}(v_1, v_2, v_3) \leftrightarrow \\
&\quad (\exists y.\, v_1 = [] \wedge v_2 = y \wedge v_3 = y) \vee \\
&\quad (\exists wxyz.\, v_1 = [w|x] \wedge v_2 = y \wedge v_3 = [w|z] \wedge \text{append}(x, y, z))
\end{aligned}$$

This is logically equivalent to the conjunction of the clause soundness and clause completeness axioms for append/3.

To illustrate the procedure for function definitions, we will of course look at the function length/1. We start with the clauses in their $\psi \leftarrow \phi$ form, as before:

$$\text{length}([]) = 0 \leftarrow \mathit{true}$$
$$\text{length}([x|y]) = 1 + \text{length}(y) \leftarrow \mathit{true}$$

The clause soundness axioms are therefore:

$$\text{length}([]) = 0 \leftarrow \mathit{true}$$
$$\forall xy.\ \text{length}([x|y]) = 1 + \text{length}(y) \leftarrow \mathit{true}$$

No quantifier is required on the first of these, since there are no free variables. To get the combined formula, we introduce fresh variables v_1 and v_r, and define:

$$\begin{aligned}
\psi' &\equiv \text{length}(v_1) = v_r \\
\phi'_1 &\equiv v_1 = [] \wedge v_r = 0 \\
\phi'_2 &\equiv \exists xy.\, v_1 = [x|y] \wedge v_r = 1 + \text{length}(y)
\end{aligned}$$

$$\forall x y_1 y_2 z.\ \text{append}(x, y_1, z) \wedge \text{append}(x, y_2, z) \rightarrow y_1 = y_2$$
$$\forall z. \exists x y.\ \text{append}(x, y, z)$$
$$\forall x y. (\exists z.\ \text{append}(x, y, z)) \wedge$$
$$(\forall z_1 z_2.\ \text{append}(x, y, z_1) \wedge \text{append}(x, y, z_2) \rightarrow z_1 = z_2)$$

Figure 3.7: Mode-determinism assertions for three modes of append/3.

Putting these together we get the following clause completeness axiom:

$$\forall v_1 v_r.\ \text{length}(v_1) = v_r \rightarrow$$
$$(v_1 = [] \wedge v_r = 0) \vee$$
$$(\exists x y.\ v_1 = [x|y] \wedge v_r = 1 + ctlength(y))$$

As before, the function completion is the same as the clause completeness axiom with the implication replaced by a bi-implication.

3.7.7 Mode-determinism assertions

Modes and determinisms play a significant role in helping people understand Mercury code. They also play a part in the declarative semantics. For modes that are `det` or `multi` we generate an axiom that says that for every value of each of the inputs, there exists a solution. For modes that are `det` or `semidet` we generate an axiom that says that for every value of each of the inputs, there is at most one solution.

For example, consider the following modes for append/3:

```
:- mode append(in, out, in) is semidet.
:- mode append(out, out, in) is multi.
:- mode append(in, in, out) is det.
```

These three modes will generate the three axioms shown in Figure 3.7, respectively. Note that the third axiom, for the `det` mode, is the conjunction of the axioms that would apply for the `semidet` and `multi` modes.

One consequence of these axioms is that if a predicate has a mode where all arguments are inputs and where the determinism says it cannot fail, then any call to that predicate is logically equivalent to *true*. Unless a strict operational semantics is used, be aware that in most cases the compiler will optimize away such calls. If that happens then exceptions may not be thrown, trace goals may not be run, etc.

Similar axioms are generated for function modes that cannot fail. The axiom generated for a function's default mode states that the function is total—a vacuous statement in classical logic since function symbols are always interpreted as total functions. On the other hand, this presents a problem for functions that are `semidet`

in the forward mode, that is, the mode with all arguments as inputs and the return value as an output. These denote *partial* functions, which cannot be directly represented in classical logic. We will discuss in Section 6.2 how these function modes are handled.

3.8 Classical semantics

3.8.1 Universes

A *universe* U is a non-empty set of *values* representing the domain of discourse.[2] The idea is that terms in the syntax correspond to values in U. If a term t corresponds to a value u in a given interpretation, we say that t denotes u, and that u is the denotation of t. For example, the code in Section 2.7 implementing arbitrary precision integers as lists of digits could have the set of all integers as its universe. In this interpretation, each list of digits would denote an integer.

The examples in Section 2.1 made use of Herbrand interpretations. In these the universe is the *Herbrand universe*, which is defined as the set of all ground data terms, and each ground data term simply denotes itself. For first order code the Herbrand universe can be considered as good as any other, since, given an interpretation in any other universe U, there exists a unique map from the Herbrand universe to U that ultimately leads to the same result.

In the following discussion, for a universe U we will make use of the sets U^n for $n \geqslant 0$, where U^n is defined as the set of n-tuples of elements in U. These sets represent the possible argument values for predicates and functions of arity n.

3.8.2 Assignments

An *assignment* over a universe U is a mapping from variables to values in U. In the following we will use σ and ρ to stand for arbitrary assignments. For a variable x, the value that it maps to under σ is written as $\sigma(x)$.

If σ_1 and σ_2 are assignments such that $\sigma_1(v) = \sigma_2(v)$ for all variables v other than x, then we say that σ_1 differs from σ_2 only at x. Note that it can be the case that $\sigma_1(x)$ does still equal $\sigma_2(x)$. We write $\sigma\{v_1 \mapsto u_1, v_2 \mapsto u_2, \ldots\}$, or just $\sigma\{v_i \mapsto u_i\}$, for the assignment that differs from σ only at each of the v_i, where it maps to u_i.

It's possible to imagine applying an assignment to formula, such that all of the free variables in the formula are replaced by the values they take in the assignment. If such an assignment makes the formula true, then the assignment is thought of as a *solution*. In the next two sections we will make this concept precise.

[2]The term "domain" is also sometimes used instead of universe, but this is not quite the same as the domains that sometimes appear in the denotational semantics of other languages—there is no $\perp$ element, for example—so we will avoid using this term.

3.8.3 Interpretations

We have used the term "interpretation" a number of times to mean, intuitively, the way in which we understand the symbols appearing in a formula. For example, we have said that the symbol + can be interpreted as integer addition. The general idea of an interpretation is that it maps from syntactic elements—the predicate and function symbols—to some semantic universe.

Formally, an interpretation I over a universe U is a mapping defined on predicate and function symbols, such that:

- If f/n is a function symbol, then $I(\mathrm{f})$ is a total function $U^n \to U$. In other words, $I(\mathrm{f})$ takes n arguments from U and returns a value from U. For a constant a, $I(\mathrm{a})$ is just an element of U.
- If p/n is a predicate symbol, then $I(\mathrm{p})$ is a predicate (that is, a relation) over U^n. That is, $I(\mathrm{p})$ takes n arguments from U and returns either *true* or *false*.

Thus, the function $I(\mathrm{f})$ is the denotation of the function symbol f/n, and the predicate $I(\mathrm{p})$ is the denotation of the predicate symbol p/n.

More generally, we will need to show how an interpretation as defined above can be extended to a mapping on terms and formulas. Since these may contain variables, the result will depend on how values are assigned to those variables.

Given an interpretation I and an assignment σ, we can extend I to a mapping I_σ from terms to elements of U by applying the following rules:

- If x is a variable, then $I_\sigma(x)$ maps to $\sigma(x)$.
- If f/n is a function symbol and $t_1, \ldots, t_n$ are terms, then $I_\sigma(\mathrm{f}(t_1, \ldots, t_n))$ maps to the function $I(\mathrm{f})$ applied to arguments $I_\sigma(t_1), \ldots, I_\sigma(t_n)$. For a constant a, $I_\sigma(\mathrm{a})$ just equals $I(\mathrm{a})$.

Similarly, this increasingly overloaded mapping can be extended to atomic formulas. The following rules are applied:

- $I_\sigma(\mathit{true})$ always maps to *true*.
- $I_\sigma(\mathit{false})$ always maps to *false*.
- If t_1 and t_2 are terms, then $I_\sigma(t_1 = t_2)$ maps to *true* if $I_\sigma(t_1)$ and $I_\sigma(t_2)$ map to the same value in U. Otherwise it maps to *false*.
- If p/n is a predicate symbol and $t_1, \ldots, t_n$ are terms, then $I_\sigma(\mathrm{p}(t_1, \ldots, t_n))$ maps to the predicate $I(\mathrm{p})$ applied to arguments $I_\sigma(t_1), \ldots, I_\sigma(t_n)$.

Continuing, the mapping can be extended to compound formulas by applying the following rules:

- If ϕ is a formula constructed using a logical connective, then $I_\sigma(\phi)$ maps to a truth value via the usual (classical) truth table for that connective, using the truth values of I_σ for each sub-formula.

- If ϕ is of the form $\exists x.\, \psi$, then $I_\sigma(\phi)$ maps to *true* if $I_\rho(\psi)$ is true for some assignment ρ that differs from σ only at x. Otherwise it maps to *false*.

- If ϕ is of the form $\forall x.\, \psi$, then $I_\sigma(\phi)$ maps to *true* if $I_\rho(\psi)$ is true for all assignments ρ that differ from σ only at x. Otherwise it maps to *false*.

Finally, observe that if ϕ is a closed formula then $I_\sigma(\phi)$ is always the same regardless of the assignment. We can therefore define $I(\phi)$, without ambiguity, as being equal to $I_\sigma(\phi)$ where σ is an arbitrary assignment.

For example, consider an interpretation I in which append/3 is the list append predicate and length/1 is the list length function. Let σ be an assignment such that $\sigma(z) = [1, 2, 3]$, and let ϕ be the following formula.

$$\exists xy.\ \text{append}(x, y, z) \wedge \text{length}(x) = 2$$

We can evaluate $I_\sigma(\phi)$ as follows. Let $\rho = \sigma\{x \mapsto [1, 2], y \mapsto [3]\}$. That is, ρ is the assignment that differs from σ only at x and y, where it takes the values $[1, 2]$ and $[3]$, respectively. We have that $I_\rho(x)$ maps to $[1, 2]$, therefore $I_\rho(\text{length}(x))$ is the length of the list $[1, 2]$, which is 2. Thus $I_\rho(\text{length}(x) = 2)$ maps to *true*.

Similarly, $I_\rho(\text{append}(x, y, z))$ maps to *true*, since the arguments map to $[1, 2]$, $[3]$ and $[1, 2, 3]$, respectively, and the last of these is the result of appending the first two. Given this result and the result from the previous paragraph, we can see that $I_\rho(\text{append}(x, y, z) \wedge \text{length}(x) = 2)$ maps to *true*, via the truth table for '$\wedge$'.

Finally, $I_\sigma(\phi)$ maps to *true* because ρ is an assignment that differs from σ only at x and y, which satisfies our rule for the existential quantifier.

Had we used an assignment σ' with $\sigma'(z) = [1]$, then the append operation would have been false under the assignment $\rho' = \sigma'\{x \mapsto [1, 2], y \mapsto [3]\}$, since $[1, 2]$ and $[3]$ do not append to form $[1]$. Furthermore, no such assignment ρ' would be able to make the interpretation of the append operation true, and still have $I_{\rho'}(\text{length}(x))$ being true. As such, $I_{\sigma'}(\phi)$ would have evaluated to *false*.

3.8.4 Models

An interpretation can be thought of as one individual programmer's understanding of how a program works. If the programmer has an accurate picture in their head of the observable behaviour—the inputs and outputs—then their interpretation can be said to be a *model*.

Formally, let I be an interpretation over the universe U. We say that I is a model of a set of axioms if each of the axioms evaluates to true in that interpretation (recall that axioms are closed formulas, so we do not need to specify an assignment). If I is a model of the axioms generated by a program, then we say that I is a model of the program. Customarily, the variable M is used rather than I when discussing an interpretation that is a model.

We are now in a position to give the following.

Definition 2 (Declarative semantics)**.** *The declarative semantics of a Mercury program is the collection of models of that program.*

In other words, the declarative semantics is essentially all the possible ways of thinking about the program which accurately reflect how the program behaves.

Considering once again the arbitrary precision integers example that we saw in Section 2.7, one programmer might interpret the digit lists as integers directly. Another programmer might interpret them as lists of integers that will produce the actual integers oncce a particular function is applied. As long as their individual interpretations as a whole accurately reflect the program, then both programmers stand in equally good positions from which to make valid arguments about the program. Our definition of the declarative semantics as a set of models reflects this fact.

If there exists any model at all, then there are an infinite number of possible models. However, we generally do not have to consider all models as it is possible to get the same results by considering only a particular set of interpretations. For first-order code, it is sufficient to only consider models that are Herbrand interpretations.

Even so, there can be multiple Herbrand interpretations that are models of a program. For example, consider the program `p :- p.` The completion of this program is $p \leftrightarrow p$, which is a tautology. This means that it is true in every interpretation, specifically, p could be assigned the value *true* or the value *false*, but in either case the axiom would hold so both of these interpretations are models.

More concerning is the program `p :- not p.` In this case the completion is $p \leftrightarrow \neg p$, which is a contradiction. This means that it is false in every interpretation, which means that *there are no models*. Effectively, the behaviour of the entire program is undefined, at least in the classical semantics. This can be regarded as a moot point, however, since execution of a program that calls such a predicate would not terminate. Nonetheless, it motivates the following definition.

Definition 3 (Consistency)**.** *A theory is consistent if it contains no contradictions. That is, there is no formula ϕ such that both ϕ and $\neg\phi$ are contained in the theory.*

If there is at least one model of a theory then there cannot be any contradictions, so by this definition the theory must be consistent.

In the remainder of this guide we will assume we are working with a program whose axioms we denote by Γ. We assume that there is at least one model of Γ, so the program is consistent, We will say "model" to mean a model of Γ. With that in mind we give the following definitions.

Definition 4 (Satisfaction)**.** *Let ϕ be a formula. If M is a model and σ is an assignment such that $M_\sigma(\phi)$ is true, we say that M* satisfies *ϕ* under *σ, and that ϕ is true in M under σ. If ϕ is closed, we additionally say that M satisfies ϕ, and that ϕ is true in M. If there exists any model that satisfies ϕ under some assignment, we say that ϕ is* satisfiable.

Definition 5 (Validity)**.** *Let ϕ be a formula and σ an assignment. We say that ϕ is* valid under *σ, written '$\Gamma, \sigma \vDash \phi$', if every model satisfies ϕ under σ. If this holds for every possible assignment, in particular if ϕ is closed, we additionally say that ϕ is valid, and that ϕ is a* logical consequence *of Γ. This is written as $\Gamma \vDash \phi$.*

Definition 6 (Unsatisfiability)**.** *Let ϕ be a formula and σ an assignment. We say that ϕ is* unsatisfiable under *σ if there is no model M that satisfies ϕ under σ. If this holds for every possible assignment, in particular if ϕ is closed, we additionally say that ϕ is unsatisfiable.*

Returning to the definition of solution that we gave earlier, we can say that an assignment σ is a solution to a formula ϕ if and only if $\Gamma, \sigma \vDash \phi$. That is, σ is a solution for ϕ if every model satisfies ϕ under σ.

With the above definitions, no formula can be both valid and unsatisfiable. Furthermore, if a formula is valid then its negation is unsatisfiable, and vice-versa. Not all formulas are one or the other, however: if a formula is true in some models and false in others, then it is neither valid nor unsatisfiable. For example, consider again the program `p :- p`. Because there is a model in which p is true and another model in which p is false, the formula p is satisfiable but not valid. A formula like this that is neither valid nor unsatisfiable is said to be *contingent*.

A point about the $\vDash$ notation is worth underlining. We have used this symbol to denote the validity relation between sets of axioms, Γ, and closed formulas, ϕ, and the "valid under" relation between sets of axioms, assignments, and open formulas. This symbol is commonly overloaded, in other ways, too. Some authors use it to denote the satisfaction relation between *models* and closed formulas, or between models, assignments, and open formulas. Other authors use it to denote the modelling relationship between interpretations and sets of axioms. That is, statements in the following forms may appear:

$$M \vDash \phi \qquad M, \sigma \vDash \phi \qquad M \vDash \Gamma$$

These mean, respectively, that M satisfies ϕ, that M satisfies ϕ under σ, and that M is a model of the set of axioms, Γ.

We will only need to use the forms given in our definition of validity, but the reader should be aware that the other forms may be used in other sources.

3.9 Example

Now that we have defined our semantics and specified what axioms generate a program's theory, how do we actually come up with a theorem? Theorems require proofs, so in this section we will give a couple of ad hoc arguments to try to prove that a formula is true.

Consider the formula append([1], [2], [1, 2]). We already know it is true given that append/3 is interpreted as list concatenation, but assume that we only know

$$\forall y.\ \text{append}([], y, y) \leftarrow \mathit{true}$$
$$\forall wxyz.\ \text{append}([w|x], y, [w|z]) \leftarrow \text{append}(x, y, z)$$

Figure 3.8: Clause soundness axioms for append/3.

how it is defined, and not its interpretation. Can we prove it is true using just the axioms that are generated?

For convenience, the clause soundness axioms for append/3 from Section 3.7.6 are repeated in Figure 3.8.

First attempt

One approach to proving our formula is to try to determine directly which ground atoms are contained in some arbitrary Herbrand model of the program, which we will call H. If the formula we are interested in is contained in the model, then it must be true.

Looking at the axiom for the first clause, we can see that if we assign the value [2] to y, then the formula becomes:

$$\text{append}([], [2], [2]) \leftarrow \mathit{true}$$

From this we immediately get that $\text{append}([], [2], [2]) \in H$. We are allowed to choose, as we did, any value we like for y, since y is universally quantified—the axiom is applicable to all possible choices.

Now consider the axiom for the second clause. If we assign the values [], [2] and [2] to the variables x, y and z, respectively, then the right hand side becomes $\text{append}([], [2], [2])$, which we just established is in H. If we then assign to w the value 1, the formula becomes:

$$\text{append}([1], [2], [1, 2]) \leftarrow \text{append}([], [2], [2])$$

Thus we get that $\text{append}([1], [2], [1, 2]) \in H$, which completes our proof.

Our attempt at proving a formula has been successful, but there are a lot of ways to go about building a proof, and the proof we have arrived at has a drawback. It starts by proving a small fact via the first clause, then builds a larger fact via the second clause. Such a proof is known as a "bottom-up" proof. This is an interesting way of reasoning, and in fact it reflects, to an extent, how deductive databases go about answering queries. but it does not reflect how Mercury programs are executed.

Mercury execution starts with a high level goal, and reduces it down to smaller and smaller parts until each part can be solved directly using a fact clause. This style of proof is known as "top-down", in contrast to the bottom-up proof above. We will make a second attempt at proving our formula, this time using a top-down approach.

While this will not completely describe how execution proceeds, it should help provide some intuition, and serve to motivate the operational semantics that will be the subject of the next chapter.

Second attempt

For our second attempt we will try a different strategy. We will start by assuming that the formula we wish to prove, $\text{append}([1], [2], [1, 2])$, is false. From that we will try derive a contradiction, which would show our assumption to be false, and thus prove that the formula is true. In other words, this will be a proof-by-contradiction. We refer again to the axioms in Figure 3.8.

Our approach will be to choose one of the axioms and try to match it to our formula by setting the arguments appropriately. Since the axioms are reverse implications for which we have assumed the left-hand side is false, we can infer that the right-hand side must also be false. By performing this inference a number of times in sequence we hope to reach a contradiction, that is, where the right-hand side is in fact true. We might fail to find such a contradiction, however. If we instead reach a tautology, it means we have failed to find the required contradiction. If we want to keep searching, we will need to go back to an earlier choice of axiom that we made, and choose the other axiom instead.

The predicate call we are interested in has argument values [1], [2] and [1, 2]. If we choose the axiom for the first clause, we find that there is no way to assign a value to y such we can match the argument values, because the first argument will always be [].

We therefore choose the axiom for the second clause. By assigning the values 1, [], [2] and [2] to w, x, y and z, respectively, we get:

$$\text{append}([1], [2], [1, 2]) \leftarrow \text{append}([], [2], [2])$$

We have assumed the formula on the left-side side to be false, so the formula on the right-hand side must also be false.

Trying the first axiom on this new formula, we can assign the value [2] to y to obtain:

$$\text{append}([], [2], [2]) \leftarrow \textit{true}$$

Again, the left-hand side is false so the right-hand side must also be false, but this is a contradiction because the right-hand side is *true*. We can therefore conclude that our original assumption must have been false. Thus $\text{append}([1], [2], [1, 2]) \in H$, which completes our proof.

This style of proof more closely reflects how programs are executed. It is still missing one important thing, however, which is that there were no variables in the formula we proved. Programs in general have output variables which become bound in the course of execution, so our proof technique would need to take account of that. We also want to be able to perform the inferences in a way that is less ad hoc. That, ultimately, is the aim of the operational semantics.

We will cover the operational semantics in detail in the next chapter. Before that, however, we will make some brief philosophical remarks regarding classical logic.

3.10 Philosophical remarks

The semantics we have presented is classical. It is possible this will leave the reader with the impression that Mercury is for classicists and Mercury programmers must therefore believe that classical logic is the One True Logic. This impression would not be accurate, as a lot of consideration has gone into understanding the benefits and drawbacks of classical logic, and likewise in understanding what other logics have to offer.

The motivation for the focus on classical logic is straightforward: it provides an excellent trade-off between ease of reasoning and ability to observe a broad class of bugs. Its use does not, of course, preclude the addtional use of non-classical logic. In Chapter 7 we give a non-classical meaning to the logical connectives that enables more realistic reasoning about programs and their correctness with respect to a specification. As such, it demonstrates the usefulness of "logical pluralism" as a philosophy for reasoning about computer programs.

The underlying purpose of logic is to characterize how we think, not to tell us how to think. We humans have the innate ability to judge between a good argument and a bad argument, and the best logic in a given situation is the one that most accurately reflects the way in which we exercise this ability.

Chapter 4

Operational semantics

4.1 Overview

In Chapter 3 we presented the declarative semantics of Mercury, and showed how it is possible to use the semantics to prove theorems about the program, and in particular about the solutions to formulas. The deductive system we used to construct these proofs, although its rules of inference were only hinted at informally, was essentially the standard one used with the predicate calculus.

The operational semantics of Mercury is a deductive system that, like the standard one, can be used to generate theorems. Unlike the standard one, it is based around a single rule of inference known as *SLD resolution*. This rule is able to give a top-down computational interpretation to a program, by which we mean that it defines the sequence of steps by which computation proceeds.

Generally, computation starts with a goal known as the *query*, and produces zero or more answers in the form of *substitutions*. We start this chapter by describing queries, substitutions, and unification, which are the key building blocks of the operational semantics.

We then introduce the SLD resolution inference rule, and show how *SLD trees* are defined. These give the operational semantics of "definite" logic programs, which are programs in which each clause body is a conjunction of atoms (that is, unifications, predicate calls, and logical constants).

Some important results in the meta-theory, known as soundness and completeness, will then be covered. These meta-theorems demonstrate that the declarative semantics and the operational semantics give non-conflicting views of the program behaviour.

After this we extend our semantics to deal with Mercury goals in general. We introduce the negation-as-failure rule used in SLDNF resolution, which defines the behaviour of if-then-elses and negated goals. We provide a set of structural rules to deal with other Mercury goals.

Finally, we will briefly explain the origin of the phrase "SLD resolution", which may be something that has attracted the reader's curiosity.

4.2 Queries

Queries are (usually non-ground) goals that represent the starting point of a computation. Executing the query involves finding substitutions, which we refer to as *answers*, for which each ground instance corresponds to an assignment that makes the goal valid. A query essentially asks, "What are the assignments of the free variables for which this goal is valid?"

The initial query for the execution of a Mercury program is always a single call to main/2. It will, however, also be useful to consider queries that represent subcomputations within the program. For definite logic programs, such queries can be written in the following form:

```
:- Goal1, Goal2, ..., GoalN.
```

where each `GoalI` is an atom, and commas are read as conjunction. If the conjunction of goals is in solved form, as defined in the next section, then no further computation is required: the corresponding substitution is the only answer.

The query is interpreted as the formula:[1]

$$\forall \bar{x}. \mathit{false} \leftarrow \phi_1 \wedge \ldots \wedge \phi_n$$

where $\phi_1, \ldots, \phi_n$ are the formulas corresponding to the goals, and $\bar{x}$ is the set of free variables occurring in the goals. Effectively, the query is interpreted as the *negation* of the goal, and the aim is to find all substitutions that make this negation false. Hence this is an attempt at proof by contradiction, similar to our proof from Section 3.9. That is, a proof, if found, is a refutation of the query, which in turn implies that the substitution is an answer to the goal.

Proof by contradiction may seem a circuitous way of doing things, and indeed some authors define the execution procedure directly to avoid this, but we define things this way because the resulting proof steps, when written out, have the premise on top and the conclusion underneath. This is the conventional way of writing proofs, but the choice ultimately makes no difference to the outcome.

4.3 Substitutions

A *substitution* is a partial mapping from variables to data terms, such that no variable that maps to a term occurs in any of the terms in the mapping. That is, variables on the left-hand side do not occur on the right-hand side. We write substitutions in the form:

```
{V1 = t1, ..., VN = tN}
```

[1] The origin of this notation, as with many other things in logic programming, comes from theorem provers. The list of goals on the right-hand side of '`:-`' is taken as a conjunction, as we have done. The list of goals of the left-hand side is taken as a disjunction, and since in our case the list is empty, the disjunction is equivalent to *false*. As usual, free variables are implicitly universally quantified.

where `V1` to `VN` are variables, and `t1` to `tN` are arbitrary data terms (possibly variables themselves) that the variables respectively map to. The condition is that `Vi` does not occur in `tj`, for any values of `i` and `j`.

A substitution applied to an expression `t` yields an expression which is the same as `t`, but with each occurrence of a variable that maps to a term in the substitution replaced with the mapped term. A substitution can be applied to a goal in a similar way, by replacing each free occurrence of a variable with the term it maps to, if any.

Observe that a substitution without the braces is just a goal consisting of a conjunction of unifications. Indeed, substitutions can be thought of as goals that are in "solved form", in that applying them once to an expression or goal is straightforward and is sufficient to produce the entire effect (that is, substitutions are idempotent). The aim of computation is essentially to put goals into their solved forms.

Substitutions represent the state of computation: they record all we know about the variables so far. In Mercury, the instantiatedness of a set of variables describes the possible substitutions at that point in the code, which tells us something about what form the substitution must take. If the `inst` of a variable is `free` then the variable is not mapped to anything. If it is `bound` then the variable is mapped to a term whose principal functor is one of the ones listed, and whose arguments are described by the corresponding argument `insts`. If it is `ground` then the variable is mapped to a term that contains no variables.

While substitutions bear a resemblance to the assignments that we defined in Section 3.8.2, note that assignments map variables to elements of the universe. That is, assignments are semantic in nature, whereas substitutions map variables to data terms possibly containing other variables, and are thus syntactic.

4.4 Unification

Given two possibly non-ground data terms, unification is the process of finding a substitution on variables such that applying it to either term yields the same result. A substitution that makes two terms identical in this way is called a *unifier* of those terms.

Terms do not always have a unifier, in which case we say that the terms do not unify. If they do unify, however, there is always a "most general unifier" that does the least amount of binding possible. There may be more than one most general unifier, but they will be unique up to renaming of variables. The unification algorithm aims to find one such unifier.

For example, consider the terms `f(X,g(X))` and `f(h(Y),Z)`, and the substitution:

```
{X = h(Y), Z = g(h(Y))}
```

Applying this substitution to either of the terms yields the same result, namely `f(h(Y),g(h(Y)))`, so this substitution is a unifier. It is not difficult to see that it is a most general unifier.

The unification algorithm can be seen as a procedure for putting equations into solved form, that is, in the form of a substitution that is a most general unifier. Taking the above example, if the query is:

```
:- f(X,g(X)) = f(h(Y),Z).
```

then in solved form this would be:

```
:- X = h(Y), Z = g(h(Y)).
```

which corresponds to the substitution we had above.

In general, consider a query that is a set of equations as follows, where `s1` to `sN`, and `t1` to `tN`, are arbitrary data terms (possibly variables):

```
:- s1 = t1, s2 = t2, ..., sN = tN.
```

Initially, all of the goals are marked as unsolved. We first select a goal `G` that is not marked as solved. If there is no such goal then the algorithm terminates. We then apply one of the following rules, depending on the form `G` takes.

- If `G` is `X = X` for some variable `X`, remove it.
- If `G` is `f(s1, ..., sN) = f(t1, ..., tN)` for data constructor `f/N`, remove it and replace it with the set of equations `s1 = t1, ..., sN = tN`.
- If `G` is `f(s1, ..., sN) = g(t1, ..., tM)` for distinct data constructors `f/N` and `g/M`, the algorithm fails.
- If `G` is `t = X` for some non-variable data term `t` and variable `X`, remove it and replace it with `X = t`.
- If `G` is `X = t` where `t` is a data term not containing `X`, then replace all free occurrences of `X` elsewhere in the query by `t`. If `X` occurred freely in the original query then keep `G` and mark it as solved, otherwise discard it.
- If `G` is `X = t` where `t` is a non-variable data term and `X` occurs in `t`, the algorithm fails.

After applying the appropriate rule we go back and select another unsolved goal, or else terminate if there are none.

If the algorithm terminates without failing then the query will be in solved form, and the corresponding substitution will be a most general unifier of the equations. If the algorithm fails then the equations do not have a unifier. Note that the order in which goals are selected does not matter, since the results will be equivalent irrespective of the selection order.

A single unification can be solved as a special case of the above algorithm, by starting with the set containing just that equation. For our above example, three applications of the rules gives us the equation in solved form:

```
f(X,g(X)) = f(h(Y),Z)
          ⇓
  X = h(Y), g(X) = Z
          ⇓
  X = h(Y), Z = g(X)
          ⇓
X = h(Y), Z = g(h(Y))
```

Had this query included the goal `Y = Z`, the outcome would have been different, as we would eventually reach the equation `Z = g(h(Z))` and thus we would fail due to the last rule, which is the occurs check.

The algorithm described here is originally due to Martelli and Montanari. We can extend the algorithm to also allow for function calls in the goals, not just data terms as we currently do, however we will need to handle function calls differently in order to implement semantic rather than syntactic equality. We will see how to do that as part of the resolution algorithm, in the next section.

4.5 SLD resolution

Deductive systems often use inference rules that follow a pattern of one introduction and one elimination rule for each logical symbol. This provides an elegant understanding of how the logic works, but for logic programming this view is not particularly useful since these rules do not provide any computational interpretation—they do not say how a program should be executed.

Instead, at least for logic programs not using negation, we use one main inference rule, which is known as SLD resolution.[2] Proofs in this system start with a query in the form:

```
:- Goal1, ..., GoalN.
```

Each of the goals is an atom (that is, a unification, a predicate call, or a logical constant), and the commas are read as conjunction. We assume for now that the program clauses are definite, so the body of each clause consists of a (possibly empty) conjunction of atoms. The more general case of Mercury clauses will be addressed in Section 4.11.

Each inference step takes a query and produces a *resolvent*, which is also in the form of a conjunction of atoms. The resolvent then either becomes the query for the next inference step, or the computation terminates. A sequence of resolvents that arises via this process is known as an *SLD derivation*.

As discussed in Section 4.2, a query represents an assertion that the conjunction of goals is false for all variable assignments. The aim of SLD resolution is to derive

[2]Historically, resolution was used as an inference technique in automated theorem provers. SLD resolution, which is an instance of this technique, was found to have a useful computational interpretation. It is from this that logic programming was developed.

a contradiction, thereby refuting the assertion. This occurs when the goals are in solved form, or there are no more goals left in the query. If such a contradiction is reached then the substitution corresponding to the solved goal is an answer.

The answer represents an assignment (or, if free variables remain, a set of assignments) for which the assertion is refuted, and therefore for which the conjunction of goals is valid. We refer to a derivation that results in a contradiction, and the substitution that it produces, as a *success*. We refer to the assignment or set of assignments that the answer represents as a *solution*.

Conversely, if at any stage the unification procedure fails then the derivation has reached a tautology. This means that we have failed to find a refutation, and we refer to this case as *failure*.

The third possibility is that neither a contradiction nor a tautology is found. We refer to this case as *nontermination*, but note that, aside from running forever, this also includes cases of abnormal termination such as throwing an exception.

The SLD resolution algorithm is parameterized by a selection function that returns a selected goal based on the current and previous queries. As before we will mark some goals as solved as we go, and require the selection function to choose a goal that is as yet unsolved. If the selected goal contains any function calls then the selection function also returns a selected function call from the goal.

The algorithm proceeds as follows. If there are no unsolved goals, the derivation succeeds. Otherwise, select an unsolved goal `G` using the selection function. Apply one of the following rules, depending on the form `G` takes.

- If `G` is `true`, delete it.

- If `G` is `false`, the derivation fails.

- If `G` is a unification between two data terms, handle it according to the rules given in the last section for the unification algorithm. If that fails then the derivation fails.

- If `G` is an atom that contains a function call, and the selected function call in that goal is `f(t1,...,tN)` for a function `f/N`, then choose a clause whose head takes the form `f(s1,...,sN) = sR`. Rename variables as necessary so they do not conflict with any variables already present in the query. Remove the selected function call and replace it with `sR`, and to the query add the unifications `t1 = s1, ..., tN = sN`, followed by the clause body if present.

- If `G` is a predicate call `p(t1,...,tN)` for a predicate `p/N`, then choose a clause whose head takes the form `p(s1,...,sN)`. Rename variables as necessary so they do not conflict with any variables already present in the query. Remove the selected predicate call and replace it with the unifications `t1 = s1, ..., tN = sN`, followed by the clause body if present.

After applying the appropriate rule we go back and select another unsolved goal. If there are no such goals, the derivation succeeds.

To illustrate the algorithm, consider the call `append([a,b],[c],Xs)`. We will produce the derivation that results from a particular sequence of clause choices. For convenience we repeat the clauses for `append/3` here:

```
append([], Bs, Bs).
append([V | As], Bs, [V | Cs]) :-
    append(As, Bs, Cs).
```

The initial query is as follows:

```
:- append([a,b], [c], Xs).
```

We start by choosing the second clause. We rename the clause's variables by adding a numerical suffix, and replace the call in the query with argument unifications and the renamed clause body. This results in the following resolvent:

```
:- [a,b] = [V1 | As1], [c] = Bs1, Xs = [V1 | Cs1],
   append(As1, Bs1, Cs1).
```

After selecting each of the unifications and applying the unification rules, we get:

```
:- Xs = [a | Cs1], append([b], [c], Cs1).
```

We again choose the second clause. We rename the clause variables with a different suffix and replace the call as before, which results in:

```
:- Xs = [a | Cs1], [b] = [V2 | As2], [c] = Bs2,
   Cs1 = [V2 | Cs2], append(As2, Bs2, Cs2).
```

Running unification rules again, we get:

```
:- Xs = [a,b | Cs2], append([], [c], Cs2).
```

This time we can choose the first clause, resulting in:

```
:- Xs = [a,b | Cs2], [] = [], [c] = Bs3, Cs2 = Bs3.
```

Running unification rules one last time we end up with:

```
:- Xs = [a,b,c].
```

which is the answer to the query.

Substitutions do not necessarily end up ground, as happened in this case, but in typical Mercury usage the modes will require that they do. In any case, for each ground instance of the answer, the derivation proves that the query formula is valid under the assignment corresponding to that ground instance. This motivates a definition to end the section.

Definition 7 (Provability)**.** *Let ϕ be a formula. We say that ϕ is* provable*, written $\Gamma \vdash \phi$, if there is some goal G with a successful derivation giving substitution S, such that ϕ is the formula corresponding to G with S applied to it. Furthermore, if $\Gamma \vdash \phi$ then $\Gamma \vdash \forall x.\, \phi$ for any variable x.*

4.6 SLD trees

The rules we have given are nondeterministic, in the sense that the order in which goals and clauses are selected is not fully specified. For the unification rules the order does not matter as the procedure will always lead to the same result, regardless of the choices made. The SLD resolution rules, on the other hand, require a bit more attention.

When it comes to selecting a goal, Mercury's selection function chooses a goal or function call whose initial `insts` are satisfied by the argument terms. In the strict sequential semantics, the leftmost goal that satisfies this condition is selected. In the strict commutative semantics the selected goal need not be the leftmost one, but if there are any goals that were expanded from the body of a clause then one of the most recently expanded goals is selected.

For predicate and function calls, the algorithm also requires us to choose a clause from the program. In contrast to goal selection, where a single choice is committed to without going back and trying alternatives, clause selection results in a "choice point" being created. After exploring the derivations that follow from one choice, if more solutions are sought then execution *backtracks* to the most recent choice point and makes a different choice. If no such choice point exists, that is, if all choices have previously been explored, then execution of the query fails.

In Mercury, the `nondet` determinism category refers to the second form of nondeterminism above, namely that relating to clause selection. Users do not need to declare the first form, as the compiler is responsible for making the choices in that regard. We will look at the second form of nondeterminism again in Section 4.10.

The collection of possible derivations arising from a query can be arranged into a tree, where derivations branch off at each choice point in accordance with execution of the query. Thus, execution involves a depth-first traversal of the tree. Trees of this form are known as SLD trees.

Figure 4.1 shows the SLD tree for the goal `append(X, Y, [1,2])`. At each node, one of the conjuncts is selected, and there is one child node for each of the possible resolvents. Where nodes have multiple children due to multiple clauses being applicable (which is all non-leaf nodes in this case), a choice point is pushed onto a stack so that the tree can be traversed in a depth-first manner.

For the leftward pointing edges, the first clause was chosen, and for the rightward pointing edges, the second clause was chosen. After each clause choice, the clause variables are renamed apart, and the head variable unifications are solved as far as possible and substituted into the body. The resulting goal is the resolvent for that edge.

Leaf nodes containing solved goals are the computed answers. The black square indicates that unification of the head variables failed, thus the derivation (that is, the branch of the tree) fails. Thus, a depth-first, left-right traversal of this tree produces the following three answers:

```
{ X = [], Y = [1,2] }
```

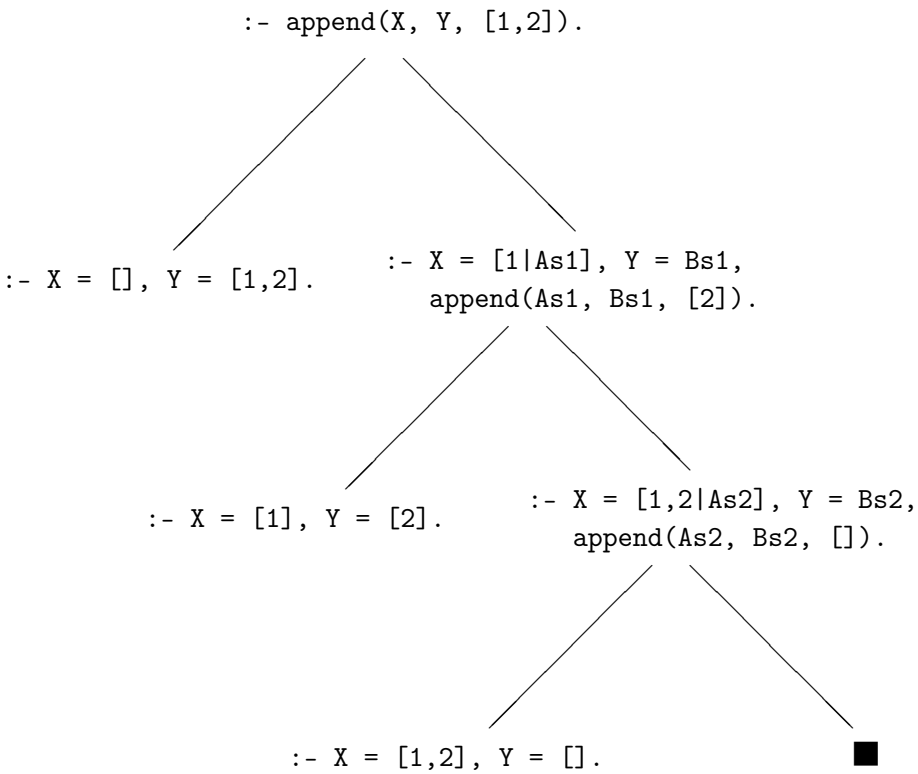

Figure 4.1: SLD tree for the goal `append(X, Y, [1,2])`. The first clause was selected for the leftward edges, and the second clause for the rightward edges. Leaf nodes with solved goals are the computed answers. The black square indicates failure.

```
{ X = [1], Y = [2] }
{ X = [1,2], Y = [] }
```

There are no choice points remaining once the black square is reached, so if further solutions are sought after the first three then the query as a whole will fail.

Like derivations, SLD trees can be considered successful, failed, or nonterminating. Since there may be multiple derivations to consider the correspondence is not direct, but is defined as follows:

- If *any* of the derivations are successful, then the tree is successful.
- If *all* of the derivations are failed, then the tree is failed. (This case is also referred to as "finite failure".)
- If there is at least one derivation that is nonterminating, and there are no successful derivations, then the tree is nonterminating.

A query or goal is successful, failed, or nonterminating, depending on whether the SLD tree that derives from it is successful, failed, or nonterminating, respectively. For example, the query from Figure 4.1 is successful since three answers are computed before it fails.

4.7 Soundness and completeness

In the course of this chapter and the previous one a number of concepts have been introduced, some of which are essentially declarative in nature, others operational. In many cases the concepts come in pairs, one corresponding to the declarative view and the other to the operational view, which nonetheless reflect the same underlying concept.

Figure 4.2 shows some of the correspondences between declarative concepts and their operational counterparts. Of particular importance is that between validity and provability, as that provides the basis for many of the other equivalences. The relationship between them is expressed by the following theorems.

Theorem 2 (Soundness)**.** *Let ϕ be any formula. If $\Gamma \vdash \phi$ then $\Gamma \vDash \phi$. That is, provability implies validity.*

Theorem 3 (Completeness)**.** *Let ϕ be any formula. If $\Gamma \vDash \phi$ then $\Gamma \vdash \phi$. That is, validity implies provability.*

Between them, these theorems state that there is an equivalence between truth as expressed in the model, and truth as expressed by the program execution.

A deductive system like this, for which soundness and completeness holds, is sometimes referred to as a "full logic". In this context the declarative view is referred to as model-theoretic, while the operational view is referred to as proof-theoretic. The equivalence between the model-theoretic and proof-theoretic, as established by the theorems, can be expressed by the somewhat cute formula $\vDash \equiv \vdash$.

Declarative concept		**Operational concept**
values	$\longleftrightarrow$	ground data terms
equality	$\longleftrightarrow$	unification
assignment	$\longleftrightarrow$	substitution
solution	$\longleftrightarrow$	answer
truth	$\longleftrightarrow$	success
falsity	$\longleftrightarrow$	failure
existence of model	$\longleftrightarrow$	consistency
validity	$\longleftrightarrow$	provability

Figure 4.2: Correspondences between declarative and operational concepts.

Of practical significance to programmers is that they can freely switch between thinking declaratively and thinking operationally. As we have seen, the former can allow for much simpler reasoning about programs than the latter. But, as we have also seen, this can only go as far as reasoning about partial correctness—there will always be situations where the programmer needs to reason operationally in order to verify correctness. Being able to freely switch between the two means programmers can use the declarative semantics most of the time, but can temporarily switch to the operational semantics when that is required.

It is, therefore, this correspondence between declarative and operational notions that justifies the use of the dual semantics of declarative programming, above and beyond the operational semantics that programming languages in general provide. In other words, we are justified in having two horizontal arrows in the middle and lower part of Figure 1.1 on page 2, instead of one.

4.8 Operational incompleteness

At first glance, the Completeness theorem appears to put us in a kind of programmer's utopia. All that is required, it seems, is for the programmer to specify the logical outcomes they want, and the deductive system will know what to do.

Unfortunately, and perhaps unsurprisingly, this is not the case. A careful examination of the Completeness theorem shows that what the theorem states is that, if a formula is valid, there *exists* some proof that can be reached via application of the resolution rule. The resolution rule, however, is not deterministic, and while execution nominally involves making all possible clause selection choices eventually, the compiler still has to commit to a particular clause ordering, as well as needing to commit to a particular goal selection at each stage. If the wrong choices are made, then a nonterminating derivation may be explored when there is in fact a successful or failed branch that would have been reached with different choices.

The code in Figure 4.3 illustrates this point. The predicate `p/0` has two clauses,

```
p :- p.          q :- q, false.
p.
```

Figure 4.3: Two predicate definitions that illustrate issues with completeness.

the first of which immediately loops. If clause selection chooses this clause first for every call then the program loops indefinitely without succeeding. The second clause, however, proves that p is valid. As required by the Completeness theorem, the proof of this validity does exist—execution needs only to select the second clause at some stage—but with the above clause selection this proof will never be reached.

Similarly, the predicate `q/0` has a single clause whose body is comprised of two conjuncts. The first conjunct immediately loops, so if goal selection chooses this goal first for every call then the program loops indefinitely without failing. The second conjunct proves that q is unsatisfiable, and similarly to the previous example the proof would have been found if the second conjunct was selected at some stage.

For the purposes of programming, this situation is effectively a form of incompleteness, despite the theorem that says otherwise. This is why the Mercury Language Reference Manual talks about implementations being "at least as complete as" the strict commutative semantics. The term "complete" here refers to the form of effective completeness discussed in this section, rather than that discussed in Section 4.7.

The concept of completeness is used in many ways in mathematics. Indeed, in Section 2.5 we referred to clause completeness, which is another distinct usage of the word. It is thus helpful, in the context of logic programming, to explicitly refer to the effective form of completeness discussed in this section as *operational completeness*. The reader should be aware, however, that other authors commonly use the terms unqualified, which may cause confusion in some cases.

4.9 Negation-as-failure

So far we have been discussing definite logic programs, that is, programs without negation. This means programs without conditionals either, since they use a form of negation. In fact conditionals are more fundamental in Mercury, as '`not G`' is a shorthand for '`if G then false else true`'.

In the operational semantics, the rule for implementing negation is known as negation-as-failure. The principle is easy to understand: if a goal succeeds then the negation of that goal should fail, and similarly, if a goal fails then the negation of that goal should succeed. We can implement negation-as-failure by adding an extra rule to the SLD rules from Section 4.5; a system with this additional rule is known as SLDNF resolution.

Assuming that `G` is the selected goal, the additional rule for negation-as-failure is as follows:

- If `G` is a conditional goal of the form '`if GC then GT else GE`', then execute `GC` as a new query. If the query succeeds with a substitution `S`, replace `G` with the result of applying `S` to `GT`. If the query fails (that is, after exhausting all possible choices), replace `G` with `GE`.

Note that if the condition succeeds, it may leave choice points behind. These will lead to alternative substitutions being applied to `GT`, leading to different derivations.

Soundness of the negation-as-failure rule can be established if and only if the condition of the if-then-else does not cause any non-local variables to become instantiated. That is, resolving the condition should not lead to any variables that occur outside of the condition or then-branch to appear on the left hand side of an equation in the substitution. This requirement can be challenging to verify manually. Fortunately, Mercury's mode system tracks changes to variable instantiation, so the compiler is able to perform the check at compile time without manual assistance.

Completeness of the negation-as-failure rule does not hold in the classical semantics. To see this, consider the following program:

```
p :- ( q ; not q ).
q :- q.
```

There is neither a successful nor a failed derivation of `q`, since its truth value is contingent on the model. Therefore, there is no successful derivation for `p` using negation-as-failure, even though `p` must be true in every model. As with inconsistent programs, incompleteness in this case is moot since the program would not terminate if run.

4.10 Committed-choice nondeterminism

One of the forms of nondeterminism in the SLD resolution algorithm, as we discussed earlier, comes about because the algorithm needs to make a choice of which clause to apply. In order to be operationally complete the algorithm leaves behind a choice point after each choice, so that execution can later backtrack to that point in order to try the other clauses.

There are two situations in which choice points are not required. The first occurs when it can be determined that execution is operationally complete without the need for backtracking. This happens if there is a goal with no output variables: in this case all answers are equivalent, so if *any* successful derivation is found then completeness is achieved. Once success occurs, any choice points created during execution of the goal are therefore no longer needed, so they are pruned away. The action of pruning away choice points is referred to as a "commit".

For example, consider the following code:

```
check(!IO) :-
    ( if p(_Out) then
```

```
        Res = yes
    else
        Res = no
    ),
    io.write_line(Res, !IO).
```

where the predicate p/1 has the following `mode` declaration:

```
:- mode p(out) is nondet.
```

Since the output from p/1 is not used outside of the condition of the if-then-else, it follows that any answer for p/1 would lead to the same answer for the if-then-else as a whole, namely, the one in which `Res` is `yes`. Any choice points created by p/1 are therefore pruned.

Given that completeness is not affected in this situation, we can conclude that there really is only ever one solution for check/2. As such, its determinism is inferred to be `det`.

The second situation in which choice points are not required is if the programmer only requires *some* solution, and is not interested in finding any alternatives once a solution has been found. This is sometimes referred to as "don't care" nondeterminism. For example, consider the following code that is slightly different from before, in that the output from p/1 affects the result:

```
check(!IO) :-
    ( if p(Out) then
        Res = yes(Out)
    else
        Res = no
    ),
    io.write_line(Res, !IO).
```

The call to p/1 in the condition of the if-then-else may have multiple answers, each one of which leads to a different answers for the if-then-else. Therefore, in this case, check/2 is inferred to be `multi`.

This is not a valid determinism for the predicate, since there is an I/O state whose uniqueness must be preserved, and backtracking into the predicate will cause uniqueness to be lost. The compiler will produce an error message to this effect.

If, however, the programmer considers all answers to be equally good (that is, they don't care which answer they get), then backtracking is not actually wanted. It can be avoided by declaring check/2 to have a determinism of `cc_multi` as follows:

```
:- pred check(io::di, io::uo) is cc_multi.
```

This declaration indicates that the predicate is "committed-choice", which means that only the first solution will be sought. In other words, any choice points created by the call to p/1 will be pruned away, and backtracking over the I/O will not be required. The code is therefore accepted by the compiler.

To support the committed-choice determinism categories, the compiler ensures that no goal with such a determinism occurs at a point in the code that is reachable via backtracking. One implication of this is that, if committed-choice nondeterminism is introduced by a goal, one of the following three things must hold:

- The compiler determines that it can perform a commit after some ancestor goal, prior to any possible failure occurring.
- The programmer uses a "promise" on some ancestor goal, prior to any possible failure occurring, to state that all solutions are equivalent. This allows the compiler to perform the commit without further verification.
- The main/2 predicate has determinism `cc_multi`.

In the third case, the program as a whole commits to one of the possible solutions, that is, final I/O states, that is defined by the declarative semantics.

Given that none of the code in question can be reached via backtracking, we can infer that the behaviour is still consistent with respect to a declarative semantics. As such, even though choice points would be required for operational completeness, the code is still pure and the programmer is still able to reason about the program declaratively.

Effectively, committed-choice nondeterminism allows the programmer to *intentionally* introduce operational incompleteness, in controlled circumstances. As we will see later on, this can be used by a programmer to give a declarative semantics to code whose real behaviour is only defined operationally.

4.11 Structural rules

We are now in a position to give rules that cover Mercury's other compound goals. Assuming that `G` is the selected goal, the additional rules are:

- If `G` is a disjunction, choose one of the disjuncts and replace `G` with the chosen disjunct. As with clause selection, a choice point is created so that execution may backtrack to the remaining disjuncts.
- If `G` is an explicit existential quantification, the quantified variables are renamed apart in the goal so that they do not conflict with any variables already in the query. The existentially quantified goal is then removed and replaced with the renamed goal.
- If `G` is defined as an abbreviation for another goal, it is removed and replaced with that other goal. Note that this rule is essentially applied at compile-time, since the replacement occurs as part of desugaring.
- If `G` is a purity or determinism cast, it is treated as if the cast was not present and executed in the same way as other goals.

- If `G` is a trace goal, the trace condition is evaluated (at compile-time or run-time, or both). If it is true then `G` is replaced by the goal with the trace condition, otherwise `G` is removed. If an I/O state is passed to the trace goal then the appropriate substitutions are made.

- If `G` is an event goal, it is removed. Doing so triggers a user-defined debug event that may be seen in `mdb`, the Mercury debugger.

4.12 What does SLD stand for?

The reader may be curious as to what the acronym "SLD" in SLD resolution actually means. Here is an explanation based on what we have covered in this chapter.

"S" stands for Selection function. The resolution procedure is parameterized by a selection function that dictates which goal to resolve next.

"L" stands for Linear. For each step in SLD resolution, a new query is generated from the previous one only, without needing to refer to other computations. As such, the proof tree consists of a single branch. This is referred to as a linear proof.

"D" stands for definite clauses. SLD resolution applies to logic programs consisting of definite clauses.

The full resolution rule is given the acronym SLDNF, where the "NF" stands for negation-as-failure. In the presence of negation the proofs are no longer linear, since they include sub-computations for negated goals. Also, obviously, the clauses are no longer definite. So perhaps SLDNF is a bit of a misnomer and only really makes sense in historical context, but nonetheless the name has stuck.

Chapter 5

The execution algorithm

5.1 Run-time unification

The abstract unification algorithm given in the previous chapter provides an overall view of the steps involved in solving unifications. Since Mercury code is compiled, however, many of these steps are able to be performed at compile-time and are thus not necessarily a major concern of the programmer.

In this section we describe the unification steps that are performed at run-time, which can be thought of as the residual steps left over after the compiler has done as much of the work as possible. This will allow programmers to get a better understanding of what kind of processor instructions will be needed, and how memory will be allocated and accessed.

The residual steps correspond to primitive unifications involving at most one function symbol, and which take the form `Y = X` or `Y = f(X1, ..., XN)`. These unifications are categorized further based on the results of mode analysis, which can infer either side of the equation as having mode `in`, mode `out`, mode `unused`, or some other mode.

Four categories of primitive unfications are compiled into inline code in the target language, which means they are executed with minimal overhead. The categories are as follows:

- Assignment unifications. These are instances of `Y = X` where one of the sides has mode `in` and the other has mode `out`. If `Y` is the output variable then we indicate such unifications as `Y := X`.

- Test unifications. These are instances of `Y = X` where both sides have mode `in`, and the type is a type constant (such as `int`, for example). We indicate such unifications as `Y == X`.

- Construction unifications. These are instances of `Y = f(X1, ..., XN)` where `Y` has mode `out` and each `Xi` has either mode `in` or mode `unused`. We indicate such unifications as `Y := f(X1, ..., XN)`.

```
append(As, Bs, Cs) :-
    As == [],
    Cs := Bs.
append(As, Bs, Cs) :-
    As == [X | As0],
    append(As0, Bs, Cs0),
    Cs := [X | Cs0].
```

Figure 5.1: The forwards mode of `append/3`, with unifications expanded and categorized as assignments, tests, constructions, and deconstructions.

- Deconstruction unifications. These are instances of `Y = f(X1, ..., XN)` where `Y` has mode `in` and each `Xi` has either mode `out` or mode `unused`. We indicate such unifications as `Y == f(X1, ..., XN)`.

Other instances of unification that are permitted by Mercury are compiled into calls to out-of-line predicates whose code is automatically generated by the compiler.

We can write a version of a predicate with unifications fully expanded (including head argument unifications). In a given mode of the predicate, we can indicate which category the unification is inferred to be in using the notation above.

Figure 5.1 shows how this would look for the forwards mode of `append/3`. The first clause performs a test on `As`, before assigning the value of `Bs` to `Cs`. The second clause deconstructs `As` into component arguments, makes a recursive call, then constructs `Cs` from the result.

5.2 Term representation

Generally speaking, a value in Mercury occupies a word, and possibly also an array of words allocated on the heap. Constants such as integer literals are stored in the word directly, whereas for terms built via a data constructor with arity > 0, the word contains a pointer to the heap array, which has one word for each argument.

(This is not the whole story. Some constructor arguments can be packed together more efficiently than indicated here, but to a first approximation this gives a reasonable indication of how much memory a term requires.)

Data constructors are represented by primary and/or secondary tag values, the former of which is stored in the pointer's unused low-bits and the latter of which is stored on the heap in an extra array element, or in the word's high-bits if it does not require a pointer.

The primitive unifications involve the following steps:

- For an assignment unification `Y := X`, we just need to copy the word from the location of `X` to the location of `Y`. If the word contains a pointer to a heap array, this array will be shared between both variables.

- For a test unification `Y == X`, we test the words for equality. If they are not equal, and if the words contain pointers to a heap array, we test that the tags are equal, and recursively test the arguments.
- For a construction unification `Y := f(X1, ..., XN)`, we allocate an array on the heap to hold the arguments and a secondary tag if required. We then fill in heap slots for each of the `Xi` with mode `in`, and the secondary tag if present. The word representing `Y` is the heap pointer, with a primary tag stored in the low-bits.
- For a deconstruction unification `Y == f(X1, ..., XN)`, we check that the tags for `f/N` are present, then use the pointer to dereference the heap slots for each `Xi` with mode `out`.

5.3 Switches

Chapter 6

Extensions

6.1 Higher-order code

In this section we show how higher-order code can be embedded in first-order logic. To do this, we need to define some additional abstract syntax, define a suitable universe, then provide a formal semantics. Here we only cover higher-order predicates; higher-order functions are handled in an analogous way.

Two additional pieces of abstract syntax are required: lambda terms which create higher-order values, and higher-order call formulas in which a higher-order value is applied to arguments. Lambda terms are written as follows:

$$\lambda v_1 \ldots v_n.\phi$$

This stands for the abstraction of ϕ over the variables $v_1, \ldots, v_n$, and corresponds to the Mercury expression:

```
(pred(V1::Mode1, ..., VN::ModeN) is Detism :- Goal)
```

If the arguments are not variables then fresh ones are introduced and unifications moved to the body, as is done with predicate completion. Higher-order calls are written as follows:

$$(t)(t_1, \ldots, t_n)$$

This stands for a call to the higher-order term t with $t_1, \ldots, t_n$ as arguments, and corresponds to the Mercury goal `(t)(t1, ..., tN)`.

In order to extend our first-order universe to handle higher-order terms, we need to add elements that the lambda terms denote. As with first-order predicates, lambda terms denote relations over the universe—that is, mappings from tuples to truth values—with the key difference being that these relations are also members of the universe itself.

For a first-order universe U, the set of all n-ary relations corresponds to the powerset of U^n. We might try to construct a higher-order universe by including the powerset along with the original set, then including all terms that can be constructed

from what we would then have, and so on. We would, however, inevitably end up with a set that supposedly includes its own powerset, but this would violate the theorem of Cantor that says this cannot happen. In a sense, the universe we have tried to define is too large to be considered a set. We can repair this situation by limiting our notion of powerset to only include *computable* relations.

A higher-order model constructed by limiting the powerset relation is known as a *general model*. If the powerset does not include every relation, then it cannot properly characterize the intended interpretation of second-order logic. However, for the purposes of programming language semantics, including just the computable relations ought to be sufficient to cover anything the programmer intends. A full semantics for second-order logic would be too powerful for our purposes.

With the universe we have just defined, we can give our semantics. Let t be the lambda term $\lambda v_1 \ldots v_n.\phi$ and let σ be an assignment. Given $u_1, \ldots, u_n \in U$, define σ' as $\sigma\{v_i \mapsto u_i\}$. Then $I_\sigma(t)$ is the relation such that $\langle u_1, \ldots, u_n\rangle$ maps to *true* in $I_\sigma(t)$ if and only if $I_{\sigma'}(\phi)$ is true.

Conversely, let t be any term denoting a higher-order value with arity n, and let ρ be an assignment. Given terms $t_1, \ldots, t_n$, we define $I_\rho((t)(t_1, \ldots, t_n))$ as true if and only if the tuple $\langle I_\rho(t_1), \ldots, I_\rho(t_n)\rangle$ maps to *true* in the relation $I_\rho(t)$.

We can express the above two logical equivalences more formally as follows:

$$\begin{array}{rcl} \langle u_1, \ldots, u_n\rangle \mapsto \mathit{true} & & \\ \quad \text{in } I_\sigma(\lambda v_1 \ldots v_n.\phi) & \iff & I_{\sigma'}(\phi) \text{ where } \sigma' = \sigma\{v_i \mapsto u_i\} \\ & & \\ I_\rho((t)(t_1, \ldots, t_n)) & \iff & \langle I_\rho(t_1), \ldots, I_\rho(t_n)\rangle \mapsto \mathit{true} \text{ in } I_\rho(t) \end{array}$$

Free variables in the lambda expression, that is, free variables in ϕ other than the v_i, are assigned values by σ, which is the assignment for the formula where the lambda term occurs. The v_i, on the other hand, are assigned values by ρ, which is the assignment for the formula where the higher-order call occurs.

Lambda terms are implemented with "closures", which are ground data terms that consist of a code pointer, along with a ground data term for each of the free variables in the lambda term. The ground data terms come from the substitution at the point where the lambda term is constructed—in our operational semantics, free variables in the lambda expression have substitutions applied to them by earlier unifications, in the same way that other free variables do.

The code pointer is to a piece of code generated by the compiler. The generated code represents a predicate whose arguments consist of the free variables in the lambda term, followed by the v_i variables. Implementing the higher-order call involves appending ground data terms for the higher-order call arguments to the ground data terms in the closure, then jumping to the code pointer with these ground terms as the arguments.

6.2 Partial functions

We mentioned in Section 3.6 that, while the predicate calculus requires that all functions be total, Mercury allows them to be partial in the form of `semidet` functions. Classically, every term denotes something, but for a `semidet` function applied to arguments outside the function's domain (that is, where the function application fails) nothing is denoted.

Such terms are sometimes called non-denoting terms, and a logic that allows non-denoting terms is called a free logic. We define the semantics by treating as false any atomic formula containing a non-denoting term; this approach is known as negative free logic (NFL).

In NFL, an "existence check" is required for each partial function called within an atomic formula, except those that are already of the form $y = \mathrm{f}(t_1, \ldots, t_n)$. The existence check succeeds if and only if the partial function term does actually denote something.

The existence check can be implemented by equating the function call with a fresh variable, and replacing the function call in the atomic formula with the variable we have just introduced. The original atomic formula is then replaced with the conjunction of the new variable equation and the new atomic formula, with the new variable existentially quantified.

That is, if t is a partial function call occurring in the atomic formula ϕ, v is a fresh variable, and ϕ' is ϕ with the sub-term t replaced by v, then ϕ is replaced as follows:

$$\phi \qquad \text{becomes} \qquad \exists v.\, v = t \wedge \phi'$$

If in $v = t$ the call to t fails, then t is non-denoting, so the conjunction fails as NFL required the original atomic formula to do. Thus, since $\exists$ only quantifies over values in the universe, existentially quantifying the fresh variable quite literally performs the existence check.

For example, consider the following declaration for a function that returns the nth element of a list, or fails if n is out of range.

```
:- func index(list(T), int) = T is semidet.
```

The goal

```
p(index(L, 3))
```

would be translated into

```
some [V] ( V = index(L, 3), p(V) )
```

If `L` has fewer than three elements then `index(L, 3)` is non-denoting, and the call to `p` should therefore fail. And indeed the translated goal would, since there is no value for `V` for which the formula is true.

In our operational semantics from Section 4.5, function calls are handled in such a way as to be equivalent to the above. The clause return value is used directly

instead of introducing the variable `V`, but in the end the effect is the same as the translation we have just given, since renamed variables from the clause behave the same as existentially quantified variables.

NFL has some features often considered undesirable. For example, the goal `index(L,N) = index(L,N)` is false if the index is out of range, even though syntactic identity suggests it ought to be true. We have defined equality semantically, however, in that two terms are equal if and only if they denote the same thing. As such, a non-denoting term can never be equal to another term, including itself.

Perhaps more concerning is the effect on substitutivity. Consider the following two goals:

```
index(L, N) \= a

V = index(L, N),
V \= a
```

Substitutivity would suggest that these are equivalent, but in fact if the index is out of range then the first succeeds but the second fails. The explanation is that `A \= B` is not an atomic formula, it is an abbreviation for `not (A = B)`. Any existence check for `A` or `B` needs to be put inside the negation, conjoined with the underlying atomic formula. The proper translation is thus:

```
not some [V] (V = index(L, N), V = a)
```

As a consequence of this, understanding the code requires knowing which goals are considered atomic, and which look atomic but are actually abbreviations.

If any of these effects leave an unpleasant taste, the best advice is to avoid `semidet` functions where possible and use `semidet` predicates instead. Functions are allowed to be `semidet` because such functions are the natural way to interpret field access functions where there is more than one constructor in the type. There is no obligation for users to write other functions in this way, however, and if there are such functions to be called, they can always be wrapped in a `semidet` predicate.

6.3 Exceptions

6.3.1 Catching

Mercury allows for thrown exceptions to be caught with the use of "try" goals. One way a try goal can be written is as follows:

```
try [] Goal
then Then
catch_any V -> Catch
```

Here `Goal`, `Then` and `Catch` are goals, and `V` is a variable. We will assume that none of the goals can fail, and neither can any of them succeed more than once.

(This is not the most general form of "try" goals; see the Mercury reference manual for the full specification.)

Operationally, `Goal` is executed first. If that succeeds without throwing an exception then `Then` is executed. Otherwise, if an exception is thrown, `Catch` is executed. In the end the try goal as a whole succeeds exactly once.

The declarative semantics of this code is as follows:

$$(\phi_{\text{goal}} \wedge \phi_{\text{then}}) \vee (\exists v.\, \phi_{\text{catch}})$$

This states that either ϕ_{goal} and ϕ_{then} are both true, or there is some value v (the exception value) for which ϕ_{catch} is true. In other words, there are multiple solutions to the try goal: one from the normal code and one from the exception-handling code for each possible exception value.

If execution was to be operationally complete, it would need to produce all of these solutions. Obviously, this is not what is intended: we only want the solution from the normal code if possible, otherwise a single solution from the exception-handling code for the exception that is thrown.

For this we use the technique that we introduced earlier in Section 4.10. The compiler infers the try goal to be `cc_multi`, and leaves an intentional completeness gap between the declarative and operational semantics. Thus, while declaratively all solutions are defined, operationally the try goal is defined in terms of the operational semantics of ϕ_{goal}. The behaviour is kept consistent with the declarative semantics, since the compiler ensures that backtracking will not require both the normal and the exceptional outputs to be produced.

6.3.2 Throwing

We have given a semantics for catching exceptions that allows exception handling to be embedded into the classical, two-valued logic. How do we do the same with throwing exceptions?

It might be tempting to just say that, declaratively, throwing an exception is the same as being false. In both cases, no variable bindings are produced. This does not work out, however, since an exception thrown from inside a negation should be the same as an exception thrown from outside. If exceptions are meant to be false then negated exceptions must be true, which breaks our own rule.

The required operational requirements are sufficiently clear: resolving an exception must neither succeed nor fail. There can be no problem with soundness, since success and failure are the only results that would constrain the models. This is not the case for completeness, however. In order to maintain completeness, throwing an exception cannot be considered valid as that would require success, and it cannot be considered unsatisfiable as that would require failure.

There is a third option, however, which is that throwing an exception can be considered contingent. That is, there exists a model in which it is true and another in which it is false. With this arrangement the deductive system can be seen to be

reasonably complete, since it would not be required to prove anything at all in the case of thrown exceptions.

Another way of saying this is that, declaratively, the throw predicate may be defined as follows.

```
:- pred throw(T::in) is erroneous.
throw(X) :- throw(X).
```

It is thus declaratively equivalent to a loop. Operationally, of course, it immediately returns to the closest enclosing catch rather than running forever.

One thing to be aware of when programming with exceptions is that, in some cases, the mode-determinism assertions imply that a given call is equivalent to *true*. As mentioned in Section 3.7.7, unless a strict operational semantics is used the compiler may optimize away such calls (the default semantics is strict, so this does not happen unless a non-default semantics is explicitly selected). Thus, if the call is intended to throw an exception, the exception may not end up being thrown.

The same issue can arise with the semantics we give here. Consider the following goal:

```
( if throw(X) then Y = 1 else Y = 1 )
```

Our semantics says this goal is equivalent to `Y = 1` in any given model, irrespective of whether or not throw(x) is true in that model. The compiler would therefore be justified in replacing this goal with `Y = 1`, meaning that the exception does not get thrown.

In general, however, it is not possible to verify that the two branches of the if-then-else goal have the same solutions, except by executing both of them. Thus, the operational semantics must still be incomplete, since it is not always possible to do this. This incompleteness does not matter, though, since we can reasonably assume that the programmer intends the exception to be thrown.

6.4 Types

So far we have largely ignored types, so the axioms we gave in Section 3.7 are technically incorrect. However, if we have unary predicates that correspond to each type, we can adjust the quantifiers to account for types. This process is known as *relativization*.

For example, given a type T and a variable x of this type, we can quantify the variable in the formula ϕ by writing $\forall x : T.\, \phi$ and $\exists x : T.\, \phi$. If the predicate corresponding to this type is p_T, then these formulas can be treated as abbreviations for $\forall x.\, p_T(x) \rightarrow \phi$ and $\exists x.\, p_T(x) \wedge \phi$, respectively. This will ensure that only well-typed terms will play a role in the semantics.

Chapter 7

Non-classical models

This section is not yet written. See the following for the main ideas:

NAISH, L., & SØNDERGAARD, H. (2014). Truth versus information in logic programming. *Theory and Practice of Logic Programming, 14(6)*, 803-840.

Appendix A

Glossary index

answer (pp. 52, 56) The substitution that results from a successful computation.

assignment (p. 43) A mapping from variables to values in the universe. (p. 67) A unification of the form `Y = X`, where one of the sides has mode `in` and the other has mode `out`.

atom (pp. 7, 30, 51) A unification, predicate call, or logical constant.

axiom (p. 35) A closed formula that is taken to be true without proof.

backtrack (p. 58) When executing a query, to jump to the most recent choice point, if one exists, in order to resume execution with a different choice.

bound (p. 30) A variable in a formula that is captured by a quantifier is said to be bound. (p. 53) A variable that is mapped by a substitution is said to be bound.

choice point (p. 58) A point in a computation at which a clause or disjunct selection was made, and which can be backtracked to in order to execute alternative branches.

clause completeness (p. 18) Of a predicate or function definition, having clauses that cover every possible input that has a solution.

clause soundness (p. 18) Of a clause defining a predicate or function, producing solutions that are true in the intended interpretation.

closed formula (p. 30) A formula in which there are no free variables.

closure (p. 72) The representation of a lambda term. It consists of a code pointer, along with values for the free variables in the lambda term.

commit (p. 63) To prune away choice points that are no longer required.

completeness (pp. 60, 63) A deductive system is complete if validity implies provability. Completeness, in reference to a deductive system, is not to be confused with operational completeness or clause completeness.

completion (p. 40) Conversion of a set of clauses that define a predicate or function into a single closed formula.

consistent (p. 46) A theory is consistent if it does not contain any contradictions.

constant (p. 29) A function that does not take any arguments.

construction (p. 67) A unification of the form `Y = f(X1, ..., XN)`, where `Y` has mode `out` and each `Xi` has either mode `in` or mode `unused`.

contingent (pp. 47, 75) True in some model, and false in some other model.

data term (p. 36) A term that does not contain any function calls.

declarative debugging (pp. 22–24) A debugging algorithm based on comparing the declarative behaviour of the program with the intended interpretation.

declarative semantics (pp. 7, 46) The collection of models of a Mercury program.

deconstruction (p. 68) A unification of the form `Y = f(X1, ..., XN)`, where `Y` has mode `in` and each `Xi` has either mode `out` or mode `unused`.

definite (p. 51) A clause is definite if it is either a fact, or its body is a conjunction of atomic goals.

derivation (p. 55) An initial query, followed by a sequence of resolvents inferred via the resolution rule.

failure (p. 56) An SLD derivation that terminates without producing an answer. (p. 60) An SLD that contains only failed derivations.

free (p. 30) A variable in a formula that is not captured by a quantifier is said to be free. (p. 53) A variable that is not mapped by a substitution is said to be free.

full logic (p. 60) A logic equipped with a deductive system that is both sound and complete.

ground (p. 53) A variable that is mapped by a substitution to a term containing no variables is said to be ground. A term that contains no variables is also said to be ground.

Herbrand interpretation (p. 8) An interpretation where the universe of values is just the set of ground data terms.

Herbrand universe (p. 43) The set of all ground data terms.

intended interpretation (p. 9) The interpretation of a specification.

interpretation (pp. 9, 44) A mapping from syntactic elements to the semantic domain.

logical (p. 16) Of a predicate or function, having a consistent declarative semantics across all calls; pure. (p. 28) Pertaining to the symbols that are a fixed part of the language, in contrast to the predicate and function symbols defined by the program.

missing answer (p. 18) One of the two classes of bugs that are observable in the declarative semantics. An answer is missing if it is false according to the program as written, but true in the intended interpretation. Also see *wrong answer*.

model (p. 45) An interpretation under which a set of axioms are all true.

negation-as-failure (p. 62) An inference rule that says if a goal succeeds, then the negation of that goal should finitely fail, and vice versa.

negative free logic (p. 73) A logic that permits non-denoting terms, and treats atomic formulas containing non-denoting terms as being false.

non-denoting term (p. 73) A term that contains a call to a `semidet` function, with arguments for which the function fails.

non-logical (p. 16) Of a predicate or function, not having a consistent declarative semantics across all calls; impure. (p. 28) Pertaining to the predicate and function symbols defined by the program, in contrast to the symbols that are a fixed part of the language.

nonterminating (p. 56) An SLD derivation that neither succeeds nor fails. This includes infinite derivations, as well as those which abort with an exception. (p. 60) An SLD tree that contains no successful derivations and at least one nonterminating derivation.

occurs check (p. 37) A check that a variable is not bound to a term that contains that variable. Also know as an occur check.

operational completeness (p. 61) The extent to which an operational semantics is able to avoid unnecessary nontermination.

operational semantics (pp. 51–61) The computation defined by a program.

partial correctness (p. 18) Correctness according to the declarative semantics. It does not consider issues such as computational complexity.

provable (p. 57) A closed formula for which there exists a successful derivation is said to be provable. Can be written as $\Gamma \vdash \phi$.

pure (p. 11) Of a predicate or function, having a consistent declarative interpretation across all calls, regardless of the mode in which the call is made.

query (p. 52) The initial goal for a computation.

relativization (p. 76) Adding explicit type checks that cause failure, to an otherwise untyped logic program.

resolution (p. 55) A type of inference rule. Logic programming uses SLD resolution as its primary mechanism of computation.

resolvent (p. 55) The query that results after performing an inference step using resolution.

satisfiable (p. 46) A model satisfies a closed formula if it maps the formula to *true*. If a model exists that satisfies a formula, that formula is said to be satisfiable.

semantic equality (p. 35) The relation between terms that holds if and only if both terms denote the same value.

SLD tree (p. 58) A tree formed from SLD derivations by including choice point nodes, where the derivations branching off from that choice point are the child nodes.

solution (pp. 43, 47, 56) An assignment or set of assignments under which, in every model, a formula is true.

solved form (pp. 53, 54) A goal that takes the form of a substitution.

soundness (pp. 60, 63) A deductive system is sound if provability implies validity.

substitution (p. 52) A partial mapping from variables to terms, such that no variable that maps to a term occurs in any of the terms in the mapping.

success (p. 56) An SLD derivation that produces an answer. (p. 60) An SLD tree that contains at least one successful derivation.

test (p. 67) A unification of the form `Y = X`, where both sides have mode `in`.

theorem (p. 35) A closed formula that is provable from a set of axioms. The collection of all such closed formulas is known as a theory.

unification (p. 53) The process of finding the most general substitution that makes two terms identical.

Unique Names Assumption (p. 36) The assumption that two ground data terms are equal only if they are syntactically identical.

unsatisfiable (p. 47) False in all models.

valid (p. 47) True in all models.

value (p. 43) An element of the semantic universe. Values are denoted by data terms.

wrong answer (p. 18) One of the two classes of bugs that are observable in the declarative semantics. An answer is wrong if it is true according to the program as written, but false in the intended interpretation. Also see *missing answer*.

Appendix B

Source code listings

B.1 Running example: queue ADT

```
%-----------------------------------------------------------------%
% Declarative programming in Mercury - queue ADT
%-----------------------------------------------------------------%
:- module dpm_queue.
:- interface.

:- import_module list.

    % Represents a finite sequence of elements.
    %
:- type queue(T).

    % Returns the empty queue.
    %
:- func init = queue(T).

    % put(X, Q0, Q):
    %
    % Q is Q0 with X added to the back of the queue.
    %
:- pred put(T, queue(T), queue(T)).
:- mode put(in, in, out) is det.

    % get(X, Q0, Q):
    %
    % Q is Q0 with X removed from the front of the queue.
    % Fails if Q0 is empty.
    %
```

```
:- pred get(T, queue(T), queue(T)).
:- mode get(out, in, out) is semidet.

    % unput(X, Q0, Q):
    %
    % Q is Q0 with X removed from the back of the queue.
    % Fails if Q0 is empty.
    %
:- pred unput(T, queue(T), queue(T)).
:- mode unput(out, in, out) is semidet.

    % unget(X, Q0, Q):
    %
    % Q is Q0 with X added to the front of the queue.
    %
:- pred unget(T, queue(T), queue(T)).
:- mode unget(in, in, out) is det.

    % True iff both queues represent the same sequence
    % of elements.
    %
:- pred eq(queue(T), queue(T)).
:- mode eq(in, in) is semidet.

    % Convert the queue into a list containing the same
    % sequence of elements.
    %
:- func list(queue(T)) = list(T).

%-----------------------------------------------------------------%
:- implementation.

    % queue(F, B) represents the sequence append(F, reverse(B)).
    %
:- type queue(T)
    --->    queue(
                front :: list(T),
                rev_back :: list(T)
            ).

init = queue([], []).

put(X, queue(F, B), queue(F, [X | B])).
```

```
get(X, queue([X | F], B), queue(F, B)).
get(X, queue([], B), queue(F, [])) :-
    [X | F] = reverse(B).

unput(X, queue(F, [X | B]), queue(F, B)).
unput(X, queue(F, []), queue([], B)) :-
    [X | B] = reverse(F).

unget(X, queue(F, B), queue([X | F], B)).

eq(Q1, Q2) :-
    list(Q1) = list(Q2).

list(queue(F, B)) = append(F, reverse(B)).

%-----------------------------------------------------------%
:- end_module dpm_queue.
%-----------------------------------------------------------%
```